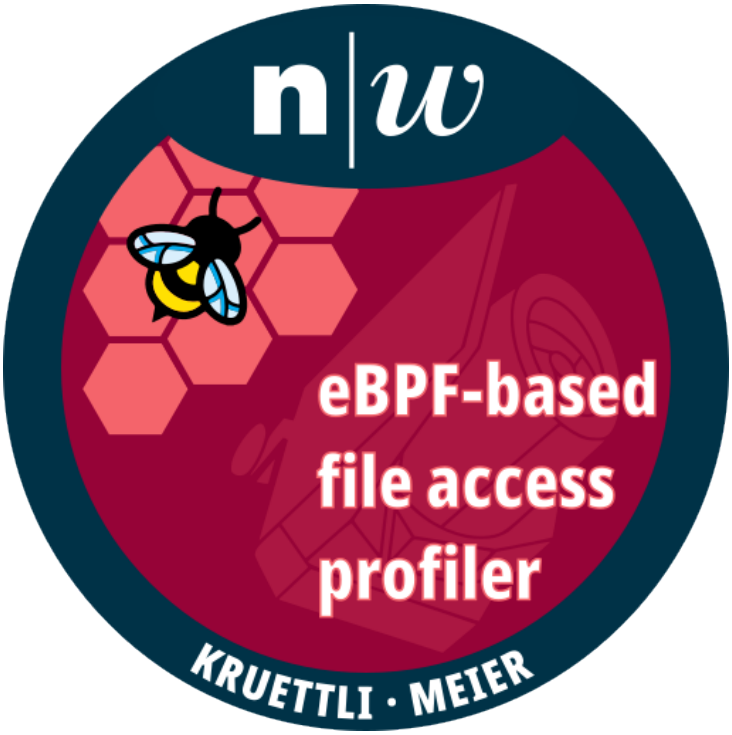
University of Applied Sciences and Arts Northwestern Switzerland
School of Engineering

Development of a specialized eBPF-based profiler for file accesses in HPC environments

Bachelor Thesis

Windisch, March 2025



Students	Philippe Krüttli Steven Meier
Expert	Simon Schubiger
Supervisor	Simon Marcin
Client	FHNW (I4DS)
Project number	24HS_I4DS28

University of Applied Sciences and Arts Northwestern Switzerland, School of Engineering

Abstract

High-Performance Computing (HPC) workloads often suffer from inefficient file access patterns, leading to severe performance bottlenecks. Traditional profiling solutions struggle to analyze file system interactions at scale due to high overhead and limited kernel visibility. This thesis presents `fsnoop`, an eBPF-based profiling tool designed to efficiently monitor and analyze file access patterns in HPC environments with minimal system impact.

The profiler is implemented using the extended Berkeley Packet Filter (eBPF) technology, enabling in-kernel tracing of file operations without modifying the underlying system or application. Key features include real-time data collection, low-overhead monitoring, and detailed pattern analysis of I/O operations such as inefficient caching, metadata bottlenecks, and overlapping file access.

To evaluate its effectiveness, `fsnoop` was tested on a real HPC workload, where it gathered almost 12 million traces allowing for in-depth analysis of the file access operations. Benchmark results indicate that `fsnoop` introduces around 2-3% CPU overhead, making it suitable for continuous monitoring in production environments.

This thesis contributes a scalable and extensible I/O profiling framework for HPC systems, providing valuable insights for optimizing file access performance. Future work includes extending `fsnoop` to support additional pattern interpretation, tracing further important file system characteristics and operations, and potentially integrating AI-driven analysis.

Keywords:

eBPF, Profiling, File Access Patterns, ESA, Euclid

Foreword

The completion of this bachelor's thesis marks the culmination of our academic education in Computer Science at the University of Applied Sciences and Arts Northwestern Switzerland. It has been both an extremely challenging but also rewarding project, providing us with valuable knowledge in systems programming using eBPF as well as experience working in HPC environments.

This thesis would not have been possible without the guidance, support, and encouragement of several individuals. First and foremost, we would like to express our deepest gratitude to our supervisor, Mr. Simon Marcin, for his insightful advice, constructive feedback, and support throughout the project. His expertise in the field has been of tremendous value.

We would also like to extend our sincere appreciation to Mr. Mark Holliman, Senior Data Systems Specialist at the Royal Observatory, Edinburgh (ROE) for providing us with the necessary tools, access, and knowledge to deploy and test our file access profiler on the DEV cluster of ROE. His prompt responses, invaluable support, and excellent cooperation were instrumental in achieving our results. Additionally, we are grateful to our peer, Tino Heuberger, for offering his support and insightful discussions throughout our closely related bachelor's theses.

Finally, we thank our families and friends for their continuous encouragement and belief in us. Their unwavering support has been a constant source of motivation throughout this journey.

We hope that this thesis contributes to improving file access profiling in HPC environment and serves as a great framework to extend in the future.

Philippe Krüttli, Steven Meier
University of Applied Sciences and Arts Northwestern Switzerland
21.03.2025

Contents

List of Figures **vii**

List of Tables **viii**

1 Introduction **1**

1.1 Background and motivation 1

1.2 Objectives and Key Questions 2

1.3 Thesis structure 2

2 State of research **3**

2.1 Cloud-focused solutions 3

2.2 I/O profiling solutions 3

2.2.1 Darshan 4

2.2.2 IOscope 4

2.2.3 iotrace 4

2.3 Comparison of options 5

3 Theory **6**

3.1 Inefficient File Access Patterns 6

3.1.1 Low cache line utilization 6

3.1.2 File fragmentation 8

3.1.3 Patterns specific to HPC environments 8

3.2 Detecting patterns using eBPF 9

3.2.1 Metrics required 9

3.2.2 Technological considerations 10

3.3 Profiling approaches 11

3.3.1 Overhead analysis 11

3.3.2 Completeness of measurement 11

3.3.3 Choice for this project 12

4 Implementation **13**

4.1 Overview 13

4.1.1 Key technologies 14

4.2 System Architecture and Design 14

4.2.1 Probes 15

4.2.2 ProbeCollector . . . 15
4.2.3 Module . . . 16
4.2.4 fsnoop . . . 16
4.2.5 CoreCordinator . . . 16
4.2.6 PatternCoordinator . . . 17
4.2.7 PatternInterpreter . . . 18
4.2.8 Visualization . . . 19
4.2.9 Templating . . . 19
4.3 eBPF profiler . . . 20
4.3.1 Hook types . . . 20
4.3.2 eBPF hooks chosen . . . 20
4.4 Data collection and storage . . . 23
4.4.1 Telemetry types . . . 23
4.4.2 Tempo . . . 23
4.4.3 Prometheus . . . 24
4.4.4 Dataobjects . . . 24
4.5 Performance and overhead . . . 26
4.5.1 Test environment . . . 26
4.5.2 Measurements . . . 26
4.5.3 Interpretation and conclusion . . . 29
4.6 Challenges and limitations . . . 30
4.6.1 Project planning . . . 30
4.6.2 Deployment on DEV-Cluster . . . 30
4.6.3 Timestamping limitations . . . 31
4.6.4 File system filtering . . . 31
4.6.5 Line of code . . . 32
4.6.6 OS limitation . . . 32
4.6.7 Data processing . . . 33
4.7 Summary . . . 33
4.7.1 Results . . . 34
4.7.2 Metadata synchronization overhead . . . 40

5 Conclusion and Future Work **41**

5.1 Summary of Key Findings 41

5.2 Discussion of fsnoop's Impact and Limitations 41

5.3 Future Work 42

5.3.1 Scalability Testing on Large-Scale HPC Clusters 42

5.3.2 Automatic Bottleneck Detection using AI/ML 42

5.3.3 Enhancing Metadata Profiling & Cache Analysis 42

5.4 Final Remarks 42

6 Usage of external tools and AI **43**

Bibliography **44**

Declaration of Honesty **47**

A Original project description **49**

B Project declaration **50**

List of Figures

1.1 Simplified overview of Euclid's data processing pipeline 1

3.1 Inefficient Loop Nesting 6

3.2 Efficient Loop Nesting 7

3.3 Schematic comparison of sampling-based vs. instrumented profiling 11

4.1 High Level Architecture of this solution 13

4.2 The modular architecture of fsnoop, illustrating its major components, including the eBPF profiler, pattern interpreters, and visualization pipeline. 14

4.3 Simplified view of a system call with kprobes and kretprobes 15

4.4 Simplified process of managing PIDs to track within `fsnoop` 17

4.5 An overview of traces collected in the Euclid test run, split by operation type . . 34

4.6 An overview of how many operations were caused by which executable during the Euclid test run 35

4.7 The average duration of each operation type in the Euclid test run 35

4.8 The average span duration over time in the Euclid test run 36

4.9 A list of some long running operations in the Euclid test run, highlighting potential bottlenecks 36

4.10 An overview of traces collected in the Euclid test run and interpreted with the lclu interpreter. Relationship differences to offset 37

4.11 An overview of mock traces with described areas in diagramm. Relationship differences to offset 37

4.12 An overview of traces collected in the Euclid test run and interpreted with lclu interpreter. Relationship sequential count and difference 38

4.13 An overview of mock traces with described areas in diagramm. Relationship sequential count and difference 38

4.14 An overview of traces collected in the Euclid test run and interpreted with lclu interpreter . difference over time 38

4.15 An overview of mock traces with described areas in diagramm. Difference over time 38

4.16 An overview of traces collected in the Euclid test run and interpreted with overlapping interpreter . Real data 39

4.17 An diagramm of traces from one file collected in the Euclid test run and interpreted with overlapping interpreter 39

4.18 An overview of traces collected interpreted with overlapping interpreter . Mock data 39

4.19 The change in summed up metadata synchronization overhead (as calculated in attribute `metadata_relevance` of a trace) over time 40

List of Tables

2.1 Comparison of `fsnoop` with existing profiling tools 5
4.1 eBPF probes implemented in `syscalls` module. 21
4.2 eBPF probes implemented in `vfs` module. 22
4.3 eBPF probes implemented in `mmap` module. 23
4.4 Description of the event object 24
4.5 Description of the `fsnoop` trace object 25
4.6 Impact of fsnoop on read latency in a lightweight task 26
4.7 Impact of fsnoop on file open and read speed 27
4.8 Impact of fsnoop on CPU-time spent in Kernel-space 27
4.9 Impact of fsnoop on I/O wait times 27
4.10 Impact of fsnoop on a simple simulated workload (file content sorting) 28
4.11 Impact of fsnoop on sequential reads and writes (*negative Overhead explained below) 28
4.12 Impact of fsnoop on a pipeline-like execution flow (*disk utilization is only really relevant when above 90% indicating a bottleneck) 28

1 Introduction

1.1 Background and motivation

In February 2024, the Euclid telescope started transmitting data back to Earth. Shortly thereafter, once the first scientific code had analyzed that data, opening a single file took up to 26 hours! But what had happened? It turns out, some of the scientific code caused inefficient file access, leading to a breakdown of the file system [1].

This project details the theory and implementation of an eBPF-based file access profiler. One use case of this solution is at the European Space Agency (ESA). Due to the large volume of data collected by ESA's Euclid mission, High Performance Computing (HPC) is essential for data analysis [2]. The scientific calculations, which are extensively I/O-bound, pose a risk of file access becoming a significant bottleneck.

The Institute for Data Science (I4DS) at the University of Applied Sciences and Arts Northwestern Switzerland (FHNW), has developed a software pipeline to execute the scientists' calculations. Performance is critical, as the pipeline operates in a distributed fashion, using a shared file system across multiple data centers in Europe and the United States.

Profiling solutions are required not only to measure performance but also to visualize bottlenecks and inefficient code sections. A separate but concurrent project focuses on CPU profiling using eBPF. This project, however, concentrates on profiling file accesses, which are crucial in Euclid's data processing workflows.

The Euclid telescope is expected to generate approximately 1 PB of initial data, which will later be enriched with information from additional sensors and telescopes. This enrichment combined with subsequent data processing cause the dataset to expand into the exabyte range. The following figure illustrates a simplified overview of this data processing pipeline.

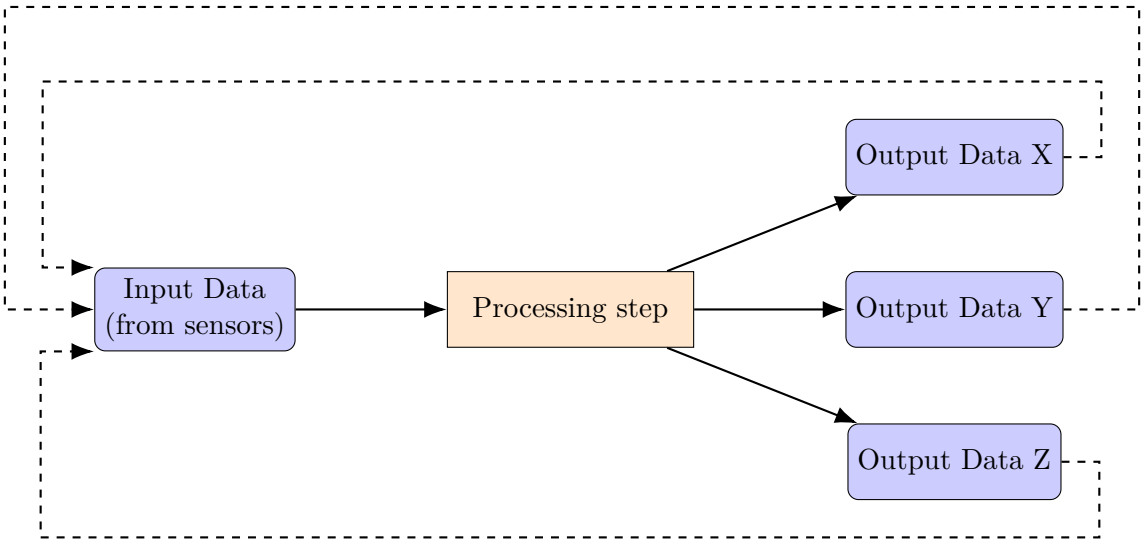


Figure 1.1: Simplified overview of Euclid's data processing pipeline

File-based input data is retrieved from Euclid and is then processed in one step. One step might have a single or multiple outputs (files). These output files will then be processed by another (or possibly also the same) processing step again, which is why the amount of data grows exponentially during processing. This highlights the importance of optimizing file access throughout the pipeline.

While the existing code performs well on scientists' local clusters, challenges arise when the code is executed on supercomputers with over 10,000 CPU cores. File system breakdowns can occur, with extreme cases like the one described above [1].

1.2 Objectives and Key Questions

To minimize the profiling solution's overhead on HPC workloads, eBPF will be used. The extended Berkeley Packet Filter (eBPF) is a technology developed in 2014 that allows for dynamic, in-kernel execution of custom code without modifying the kernel itself. Since eBPF programs run in native machine code and without needing to switch between kernel and user space, they are highly efficient [3].

The goal of the profiler is to identify the causes of I/O bottlenecks and provide insights into file access patterns in scientific code. It should help determine whether alternative file formats could improve performance and highlight inefficient code sections for optimization.

In an HPC environment, the same workload may run on multiple servers simultaneously, and the profiler must aggregate results from all nodes into a single output. User-friendly visualizations are necessary to efficiently communicate performance metrics and insights on file access frequency, process details, and access patterns.

The solution should be versatile enough for use in other HPC workloads. Although ESA's Euclid mission is the primary driver of this project, the profiler should not be tightly coupled with it.

The following key questions guide the research and implementation of this project:

- **Instrumented vs. Sampling-based Profiling:** What are their respective advantages and disadvantages?
  - Instrumented profiling triggers on specific events like read operations.
  - Sampling-based profiling triggers periodically, such as at a frequency of 50Hz.
- **File Access Patterns:** What inefficient patterns exist, and how can an eBPF profiler be used to detect them? Are certain patterns unique to HPC environments?
- **Storage Types:** How does the profiler handle RAM, SSDs, HDDs, and shared file systems using `mmap`, considering caching, network overhead, and transfer bandwidth?
- **Data Aggregation:** What methods are most efficient for consolidating distributed profiling data across nodes?
  - Multiple nodes running the same executable should result in a unified output.
  - Runs may occur over extended periods, adding complexity to data aggregation.
- **eBPF and Linux kernel:** Which Linux kernel version is required to support the necessary eBPF features?
- **CPU Overhead:** What is the CPU overhead, and is it low enough to allow for always-on profiling in production?
- **Data Representation:** How should profiling data be represented? What metrics are most relevant to scientists, and which graphical representations are most effective?

1.3 Thesis structure

Section 2 outlines the current state of research. Section 3 presents the relevant theoretical background, followed by section 4 detailing the implementation and testing results. Finally, section 5 summarizes key findings, suggests future improvements and concludes the project.

2 State of research

Modern High-Performance Computing (HPC) systems are difficult to debug and troubleshoot because only a limited amount of information can be retrieved from the kernel. These systems are characterized by their immense scale and the multitude of software they run. The HPC software targeted by this project's profiler cannot be isolated or managed using containerization technologies like Docker or traditional virtualization. This is due to the software's need for direct hardware access and its complex execution across multiple nodes. Thus, alterantive grouping and aggregation approaches must be considered [4]. There are many eBPF-based tracing tools but most of them are focused on cloud computing, an example for that is Grafana in combination with Cilium. Further research suggested that there are indeed some tools that use eBPF to trace the kernel in HPC systems, which are outlined below.

2.1 Cloud-focused solutions

eBPF is widely used in cloud environments. AWS, Apache, and Grafana all describe how they have used eBPF in their products. While the fundamental concepts are the same as in this thesis' project, the operational context differs. The system that will run this project's profiler hosts a variety of programs and applications. Other vendors' products are configured for specific cloud environment tooling, such as Kubernetes. Cilium serves as another example for the usage of eBPF. Google describes it as follows: "Cilium is an open-source project that has been designed on top of eBPF to address the new scalability, security, and visibility requirements of container workloads. Cilium goes beyond a traditional Container Networking Interface (CNI) to provide service resolution, policy enforcement, and much more." [5]

Apache SkyWalking is an open-source software, primarily designed for distributed systems. It is widely used in cloud environments but also supports distributed systems involving multiple processes. Built on eBPF, SkyWalking utilizes tracing, logging, and metrics to monitor and analyze system performance. While its documentation suggests that it can perform I/O-tracing, this capability is limited and not a main focus of the tool. Implementing I/O-profiling with SkyWalking is possible but requires significant customization, making it less suitable for projects prioritizing detailed I/O analysis. SkyWalking's Grafana integration offers a clear advantage for visualizing performance data. Its extensibility make it a versatile tool that could be adapted for certain aspects of the project. However, its design is heavily focused on cloud-based and containerized systems, less on the non-containerized, large-scale distributed HPC environments targeted in this project. Additionally, adapting the tool to analyze shared file systems and other HPC-specific requirements would likely require extensive modifications. [6] [7] [8] [9]

2.2 I/O profiling solutions

Market research brought up three interesting tools, providing profiling capabilities for HPC systems. An ideal tool would use eBPF, because it operates directly in the kernel space. This allows for system monitoring at the lowest level, gathering all relevant data for each system call without interfering with the program's execution. In addition, these tools can profile any program without requiring modifications to the source code. Futhermore, eBPF profiling tools ensure minimal performance overhead, making them ideal for continuous monitoring in resource critical environments.

2.2.1 Darshan

Darshan is the only tool in this comparison that does not use eBPF, and must be added to the application to profile at compile time or using library preloading as stated in its documentation: "Darshan instruments applications via either compile time wrappers or dynamic library preloading." [10].

While compile-time integration offers deeper profiling, library preloading provides flexibility for profiling precompiled binaries. Darshan is not intended to be used as a system-wide always-on profiling solution. Instead, an application instrumented with Darshan generates a log file. This log file provides detailed information about the I/O access patterns and other performance-related metrics. As a result, the analysis is tied to a specific application. Darshan is commonly used in HPC environments as it is focused on HPC systems and I/O profiling. As a result, it is highly optimized for these environments, offering low performance overhead. In order to have always-on analysis with eBPF, a custom-developed plugin or a similar solution would be required. [11] [12] [13] [14] [15]

Darshan is not a suitable solution for our project, as it is not eBPF-based, and requires integration at compile time or through library preloading. These limitations prevent it from meeting the client's requirements. Consequently, Darshan has been ruled out as a solution approach.

2.2.2 IOscope

Another tool developed for HPC environments is IOscope. IOscope is, as the name suggests, an I/O profiling tool for analysing and locating file access patterns. It is eBPF-based and therefore aligns well with the project goals, however, the official Git repository is outdated. The last commit to it was made on 20 April, 2018 and it remains to be checked, whether IOscope requires an update to comply with modern industry standards. Despite this limitation, IOscope is an interesting candidate for further research due to its flexibility and its focus on HPC systems. IOscope could serve as a solid foundation or source of inspiration. It includes two components designed to filter and analyze I/O activities with minimal overhead. Its structure and focus make it a useful tool for gaining insights into file access patterns in HPC environments. [16] [17]

Since the official Git repository is outdated and no longer maintained, IOscope will not form the basis of this project. However, it might serve as a source of inspiration for developing a fully-customized solution.

2.2.3 iotrace

The last promising tool is the Standalone Linux IO Tracer (iotrace). This tool was developed for, or at least in conjunction with, the Open CAS Telemetry Framework and is strongly tied to this framework. The Open CAS Telemetry Framework provides a robust foundation for managing and parsing telemetry data. A major disadvantage of iotrace is, that its functionality is limited to block file systems. Unlike IOscope, however, iotrace is still somewhat actively maintained with the last commit to the official repository being made on 18 May 2024. Furthermore, it has transitioned to using eBPF for capturing traces, enhancing its performance and efficiency. [18] [19] [20]

The standalone Linux IO Tracer (iotrace) only supports the system calls `write`, `writev`, `read` and `readv`, which provide far too few tracing points to capture all the required metrics for this project. In addition, iotrace is only officially supported on the following two Operating Systems:

- Fedora 36 (kernel 5.18.10)
- Ubuntu 22.04 (kernel 5.15.0)

All these limitations make iotrace unsuitable for this project, as its reliance on block-level I/O and OS restrictions would need to be eliminated - yielding only a minimal number of trace points in return. Then, additional work would be required to add more tracing points and enable the collection of a wider range of metrics. Given the complexity of the tool and the significant amount of time required to customize it, these efforts would not provide significant benefit. For those reasons, iotrace was also ruled out as foundation for this project.

2.3 Comparison of options

	Darshan	**IOscope**	**iotrace**	**fsnoop** (our solution)
eBPF-Based	No	Yes	Yes	Yes
HPC Optimized	Yes	Partial (outdated)	Partial (block I/O)	Yes
Overhead	Low	Unknown	Unknown	Low (∼2.5-3%)**
Real-Time Monitoring	No	No	Limited (`STDOUT` only)	Yes
Supported File Access Types	POSIX I/O*	Syscalls, VFS, mmap	Block-Level I/O	Syscalls, VFS, mmap
Requires Code Modification	Yes	No	No	No
Ease of Integration	Moderate	Difficult (outdated)	Difficult (Block-Level I/O only)	Easy (extensible, modular)

Table 2.1: Comparison of `fsnoop` with existing profiling tools

***** Darshan operates at application-level I/O, meaning it captures POSIX I/O rather than system calls or lower-level storage interactions.
****** low overhead assumed due to use of eBPF profiling, actual numbers added after tests

This comparison led to the decision to implement a custom profiler called `fsnoop`, as it is the most promising approach to meet all the requirements and desired features imposed by the client. It is eBPF-based, optimized for HPC with low overhead, and supports real-time monitoring to view output during tracing already. Furthermore, `fsnoop` supports all the needed file access types, requires no modification of the application code, and can easily be integrated into existing infrastructure and workflows.

3 Theory

3.1 Inefficient File Access Patterns

Identifying common inefficient file access patterns sets the foundation for later detecting file accesses that might lead to performance penalties. Some of these patterns are universally applicable while some might either only exist in HPC environments or have a much greater effect in those due to the different caching architecture in HPC systems.

3.1.1 Low cache line utilization

Accessing a larger block of data at a time can improve performance as the disk seek time combined with the network latency in a shared file system predominate the time required for the actual disk access. However, it is the file system's job to guarantee the optimal amount of data is actually loaded. When more data is read than requested by the user, this is called data prefetching. Data prefetching is a common caching strategy with the assumption that the user will read adjacent data next. Therefore, if data read is non-sequential, prefetching is of no use and therefore we have a below-average cache utilization [21].

The user should make use of already cached data (if the cached data will be processed at some point anyway) instead of accessing a completely different location in storage, throwing away the cache just to load the same data later on having to access the disk (and network in a shared file system) again.

Inefficient Loop Nesting

One possible cause for the abovementioned issue is Inefficient Loop Nesting. When looping over multidimensional arrays (or for that matter pictures, as they can be treated as two dimensional arrays), the storage layout has to be accounted for. Figure 3.1 shows the access pattern when the inner loop iterates over rows while the outer loop iterates over columns. Considering the file system prefetches (part of) a row (shown in red), no cache is actually used and therefore each access reads from the disk, because the subsequent read will happen on the next green block instead.

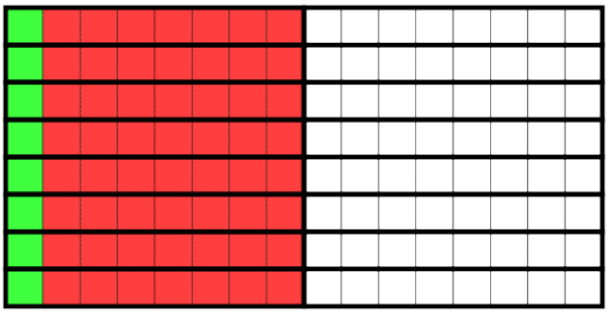


Figure 3.1: Inefficient Loop Nesting

Source: [22]

Switching the iteration argument such that the outer loop now iterates over rows while the inner loop iterates over columns results in an Efficient Loop Nesting as shown in Figure 3.2, making use of the prefetched data in the cache. [22]

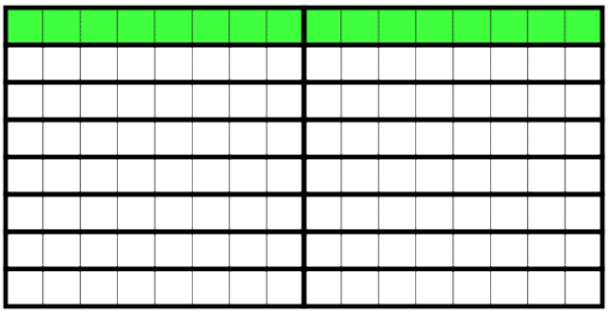


Figure 3.2: Efficient Loop Nesting

Source: [22]

Random Access Pattern

Another cause for low cache line utilization is random access. Here, cached data cannot be used since it cannot be predicted what data will be used next. Some data structures (such as hash tables) but also some algorithms (such as most graph algorithms) inherently lead to random access.

Compared to Inefficient Loop Nesting mentioned above, Random Access Pattern is often very difficult to sort out. One has to use different data structures and/or algorithms leading to major code refactorings [22].

Unexploited Data Reuse Opportunities

When working with large datasets it is often required to perform multiple actions on the same data. It is, however, crucial to apply all actions directly on the same data once that data is retrieved rather than having to re-fetch the same data later on in a second iteration. In HPC environments with a shared file system, this is even more important [22].

```
for (size_t i = 0; i < blocks.size(); i++) {
        // Read data the first time
        taskA(file.read(i));
}

for (size_t i = 0; i < blocks.size(); i++) {
        // Read data a second time
        taskB(file.read(i));
}

for (size_t i = 0; i < blocks.size(); i++) {
        // Read data a third time
        taskC(file.read(i));
}
```

Listing 1: Sample of Unexploited Data Reuse in C++

Listing 1 shows Unexploited Data Reuse, because the same chunk of data is loaded in three separate loops just to make sure `taskA` is first applied to all data, then `taskB`, then `taskC`.

```
for (size_t i = 0; i < blocks.size(); i++) {
        // Read data only once
        auto data = file.read(i);
        taskA(data);
        taskB(data);
        taskC(data);
}
```

Listing 2: Sample of improved Data Reuse in C++

Listing 2 shows how the Unexploited Data Reuse from Listing 1 can be mitigated. Just load the data first (only once) and then apply all operations (`taskA`, `taskB`, `taskC`) to this data before loading the next chunk of data.

3.1.2 File fragmentation

Data prefetching as mentioned in 3.1.1 can only be done within a single file as different files might be spread across the disk(s). Therefore, having many small files that are read and processed sequentially, could benefit from file combination as within this larger file, data from the "next file" could already be prefetched [23].

3.1.3 Patterns specific to HPC environments

The abovementioned file access patterns are all applicable both to general file systems as well as parallel file systems used in HPC environments. However, there are also inefficient file access patterns that are specific to HPC environment, and that would not occur or regular file systems.

Overlapping file accesses
In a shared file system, there are two general ways of accessing files:

- Shared file
  - Multiple processes access the same file in the parallel file system
  - Can result in concurrent access
- File per process
  - Each process gets a copy of the same file, so each access is exclusive
  - Results in higher storage demands

Concurrent access to the same file can be mitigated with e.g. byte-range locks, but when multiple processes have to access the same area in a file, this no longer works. Overlapping file accesses should be minimized as much as possible, as in those cases I/O operations will be sequential [24].

High Metadata and synchronization overhead

In parallel file systems, usually only one server stores the metadata to each file (the MDS or MetaData Server). Upon file creation, open and modification events, metadata is read and/or written. Therefore opening and closing files very often results in a bottleneck, as all parallel requests to multiple OSSes (ObjectStorage Servers) also result in a call to the single MDS. Some considerations here are [25]:

- Use larger files rather than many small ones
  - This reduces the latency-impact on I/O operations
- Limit files per directory to no more than 10.000
  - MDS has no information about the sizes of non-directory files
  - For each file, it has to aggregate data of all OSSes to e.g. sum up their sizes

Manual calls to `MPI_FILE_SYNC` or equivalent synchronization calls (when not using MPI) also result in the same behavior as frequently opening and closing files [26].

Improper file striping

Parallel file systems spread the content of a file onto multiple servers, this is called striping. Striping can help facilitate concurrent access to a single file, as multiple storage servers are involved. However, if the striping does not align properly with the data access pattern, some storage servers might be constantly over- or underutilized, resulting in bad performance [26].

MPI - Non-contiguous access without collective I/O

Multiple processes might access non-contiguous parts of the file system. By using collective I/O, one can first of all reduce the actual number of request to the file system, but you can also read larger chunks as e.g. process 1 might require only a small part of a file but process 5 might require exactly the subsequent bytes of the same file [27].

Collective I/O is an optimization technique specific to parallel file systems, so when multiple processes access those shared files without collective I/O, there is an obvious performance penalty.

MPI - Excessive use of Shared File Pointers

Shared File Pointers are file pointers that are created once per open command. They are then shared between multiple processes in a communicator group in MPI. However, this also requires synchronization, to ensure every process using this pointer always has the latest state. When relying too much on the use of Shared File Pointers, this synchronization can lead to almost sequential file accesses [26].

3.2 Detecting patterns using eBPF

3.2.1 Metrics required

File access overlap index

Measures how many overlaps happened, how long they lasted and of what type (read-read, read-write, write-write) they were. This can be implemented using metrics `File accessed`, `Process ID`, `Operation Type`, `Offset/Range`, `Timestamp`.

Metadata operations
Measures how many metadata-relevant operations happened by counting system calls (or other relevant kernel functions) resulting in metadata operations per file. These include [28]:

- `stat`, `lstat`, `fstat` that retrieve metadata about the file
- `chown`, `lchown`, `fchown` that change owner metadata about the file
- `link`, `linkat`, `unlink`, `unlinkat` that change the number of hard links to the file
- `getxattr`, `lgetxattr`, `fgetxattr`, `listxattr`, `llistxattr`, `flistxattr` that retrieve extended file attributes
- `chmod`, `fchmod`, `setxattr`, `lsetxattr`, `fsetxattr`, `removexattr`, `lremovexattr`, `fremovexattr` that change (extended) file attributes
- `write`, `writev`, `pwrite`, `pwritev`, `truncate`, `ltruncate`, `ftruncate`, `fallocate`, `sendfile` that change file size and timestamps
- `open`, `read`, `readv`, `rename`, `utime`, `utimes`, `utimensat`, `exec` that change timestamps

Read/Write latency
Measures how long `read` or `write` operations take to detect possible congestions at some storage nodes (possible due to improper file striping as mentioned above).

Number of accesses per storage block
Measures how many `read` and `write` operations happen per storage block (possible due to improper file striping as mentioned above). If striping is aligned properly with data access patterns, then it should be about the same for each storage area. If storage areas are accessed multiple times by the same process, this might indicate unexploited data reuse.

Number of cache misses
A high number of cache misses, especially in a short period of time, might be an indication of inefficient loop nesting.

I/O wait times
Measures the time the CPU has to wait for I/O-operations to complete as a fraction of the total time spent in a program or a section of it. If it is above a threshold (that remains to be defined), this might indicate either inefficient loop nesting or unexploited data reuse opportunities.

Average offset between reads
Measures the the average offset between consecutive `read` operations to detect possible random access pattern.

Delimitations of this project
MPI patterns will explicitly not be considered in this project, though they could be included at a later point.

3.2.2 Technological considerations

Since access on a parallel file system happens simultaneously from multiple nodes, identifying patterns such as concurrent access to a file cannot be implemented in the profiler itself, when the profiler is running on the worker node. Therefore, data must be gathered by all nodes during runtime and stored centrally and then afterwards, once all tracing data is gathered, it can be aggregated and patterns as well as inefficiencies can be detected.

Furthermore, in HPC environments, making use of cached data is even more important since the overhead of loading uncached data over the network is much higher than when running locally, but also because there are multiple levels of cache in those systems [29].

3.3 Profiling approaches

There are two main types of profiling:

- Instrumented profiling
  - uses event-based instrumentation
- Sampling-based profiling
  - uses a fixed or flexible sampling rate (e.g. 50Hz) to capture a snapshot of the current state

Both of these approaches present some benefits with sampling-based profiling excelling in performance, i.e. potentially lower overhead and instrumented profiling providing the benefit of capturing all data and only introducing overhead, when data is needed.

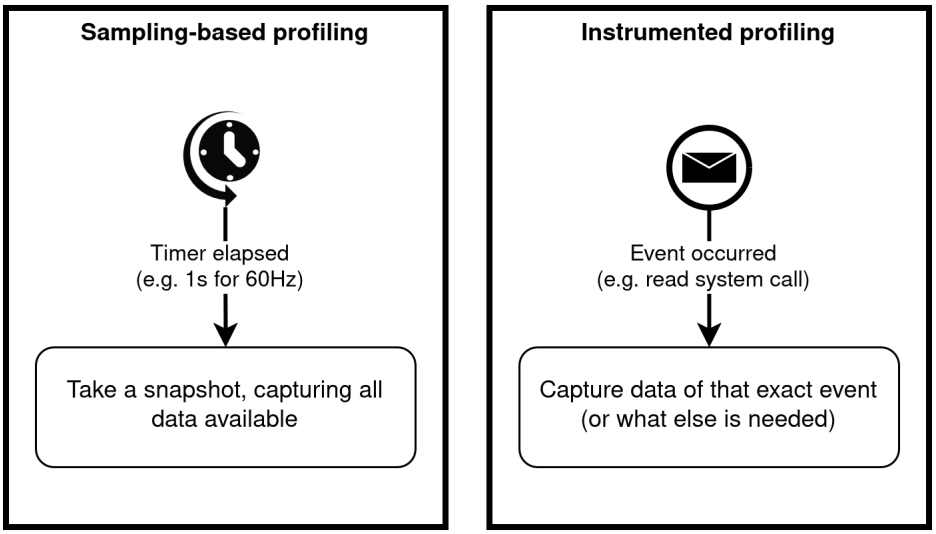


Figure 3.3: Schematic comparison of sampling-based vs. instrumented profiling

3.3.1 Overhead analysis

Sampling-based profiling has a consistent, yet also constant overhead. This means that regardless of the number of executables that should be profiled, the overhead remains the same. However, this also has the disadvantage that e.g. when no action is actually happening, because the executable to trace is waiting for I/O or some other task to finish, sampling-based profiling still generates overhead. Instrumented profiling on the other hand only generates overhead when an event, that requires profiling, occurs. However, the overhead can also grow higher when there are a lot of events happening.

This applies not only to CPU and memory overhead but also to storage overhead, as varying amounts of data may be generated.

3.3.2 Completeness of measurement

Using instrumented profiling, one will receive all the information that is needed, because every event results in a trace being generated. In sampling-based profiling, only a limited amount of data is captured, making it ideal for situations where collecting all available information would produce excessive data.

3.3.3 Choice for this project

In this project, instrumented profiling is the ideal choice. Gathering all the information, meaning every file access happening, is crucial to effectively detect patterns and inefficiencies. The overhead, while of course being important, can be neglected in terms of data generation but needs to be considered for CPU- and Memory-overhead.

Since this project focuses on events (in this case file access operations) rather than just a continuous state of a system (like resource usage over time), the solution should use instrumented profiling [30].

4 Implementation

4.1 Overview

The main focus of this profiler's design was modularity and extensibility. Since eBPF and HPC environments are rapidly evolving, this profiler was designed to be modular, extensible, and easily adaptable.

There are two major components in this architecture:

- `fsnoop` (software developed in this thesis) consisting of:
  - eBPF-profiler (gathering data on worker nodes)
  - Pattern-Interpreters (analyzing data, providing new metrics)
- Grafana with Tempo- and Prometheus-Backend (to visualize and analyze results gathered)

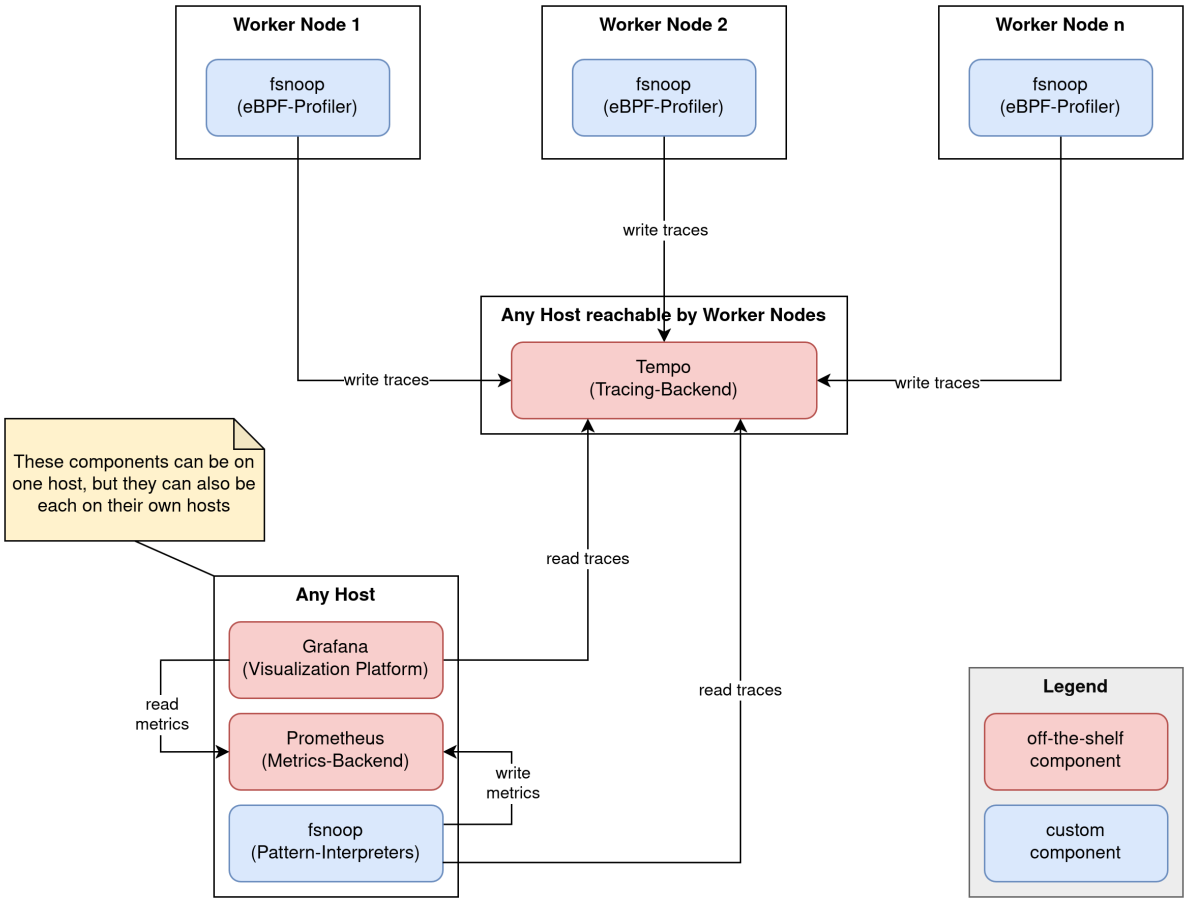


Figure 4.1: High Level Architecture of this solution

As Figure 4.1 shows, `fsnoop` is the custom component developed in this thesis and consists of both the eBPF-profiler as well as the pattern interpreters. Using off-the-shelf components like Tempo, Prometheus and Grafana ensures strong integration into existing infrastructure as well as realiable communication between hosts.

4.1.1 Key technologies

The main technologies used in this solution are:

- **eBPF/BCC**
  - for writing the profiler and hooking into systemcalls, VFS-calls etc.
  - BCC simplifies writing eBPF programs and loading them into the kernel
- **OpenTelemetry**
  - as a standard means of communication between the profiler and the tracing backend
- **Python**
  - using BCC to load eBPF program
  - to write Pattern-Interpreters with OpenTelemetry-Integration
- **Grafana**
  - to visualize and analyze the gathered data
  - Tempo as backend for Traces
  - Prometheus as backend for Metrics

4.2 System Architecture and Design

The solution architecture focuses on modularity and extensibility. All parts of the solution can be extended or replaced without affecting other components. This means that the program can continuously be developed, even after this thesis. Another advantage is, that the parts can run on separate threads or even separate hosts. This increases performance and helps keeping the overhead - especially on the worker nodes - as low as possible. The eBPF-probes that hook into kernel functions are written in C. All other parts of the program run in user space and are written in Python.

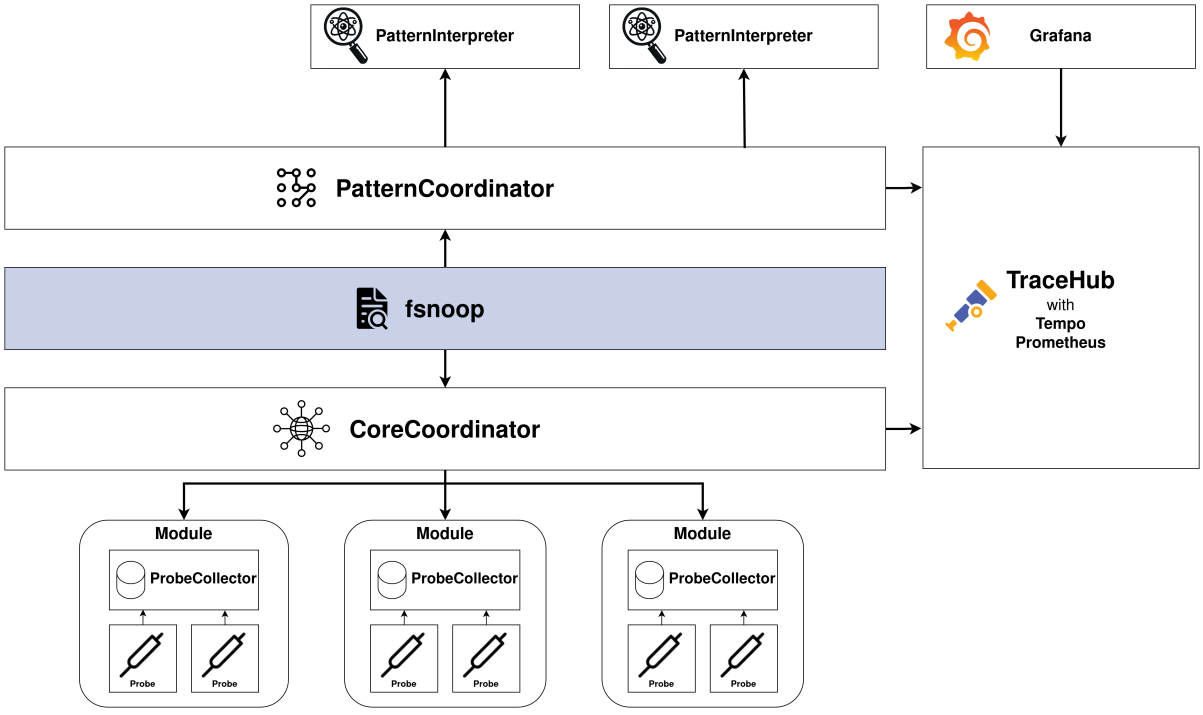


Figure 4.2: The modular architecture of fsnoop, illustrating its major components, including the eBPF profiler, pattern interpreters, and visualization pipeline.

Figure 4.2 shows the different parts of the newly developed profiling solution called `fsnoop`. The arrows visualize dependencies among components. The following sections provide a more detailed explanation of each component and its purpose.

4.2.1 Probes

The probes are one of the most important parts of the system. Their purpose is to collect information from the kernel space. An eBPF probe is developed for just one task. One example is the probe to collect information about the invocation of a read system call (`kprobe`). Another probe collects information about the exit of the read system call (`kretprobe`). They are kept as simple and small as possible, as to not generate more overhead than necessary. No business logic is included in any probe.

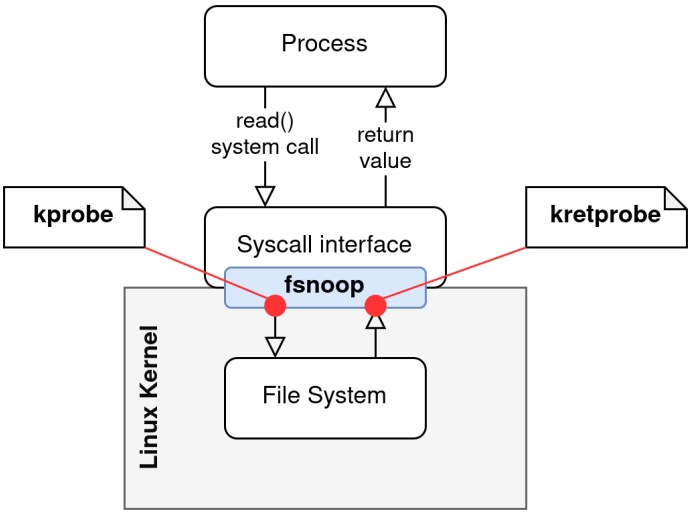


Figure 4.3: Simplified view of a system call with kprobes and kretprobes

Figure 4.3 shows how `fsnoop` intercepts the `read()` system call and where `kprobes` and `kretprobes` come into play. Each of the red dots in the figure above are probes and for every relevant system call or other kernel functions, such probes can be developed.

Probes store data about an operation within an event object (specified in 4.4.4). Data about the operation, the calling process as well as the duration are stored in this object and then passed to user space using a perf event buffer. This buffer is defined using a BPF perf event array map (of type `BPF_MAP_TYPE_PERF_EVENT_ARRAY`) - commonly named `events` - and is read from user space as a buffer, processing each event sequentially.

4.2.2 ProbeCollector

A ProbeCollector combines probes of the same type. So, for example the entry-point probe for the read system call and the exit-point probe for the same system call are combined by one ProbeCollector. This results in usually one ProbeCollector per operation type. However, in some cases it makes sense to further combine multiple highly related calls such as the `pwritev` and the `pwritev2` system calls. Then, both of them are handled by the same ProbeCollector.

4.2.3 Module

Similar ProbeCollectors are the further combined by a module. The following modules have been implemented in the current solution:

- syscalls (bundles all system calls)
- vfs (bundles all VFS calls)
- mmap (bundles all caching-related calls)

Modules are designed so that they can be added or removed to the final output via a configuration file. This allows for a highly customizable solution that can also grow and shrink over time. Modules also have the advantage that they introduce a much clearer structure into the solution, allowing the developer to find the code to adjust much more easily.

4.2.4 fsnoop

While the program developed in this bachelor's thesis is called `fsnoop`, this is also the entrypoint of the system. It parses command-line arguments, possibly throws errors and if the arguments match, they are passed to either the CoreCoordinator or the PatternCoordinator.

```
// Start fsnoop to gather data (root-privilege required for eBPF)
$ ./fsnoop

// Start fsnoop to interpret patterns for PIPELINE_ID "my-test-pipeline"
$ ./fsnoop -d -p "my-test-pipeline"
```

To see all the possible command-line arguments and their meaning, a simple usage help can be displayed using the following command:

```
// Display usage-help for fsnoop
$ ./fsnoop -h
```

4.2.5 CoreCordinator

The CoreCoordinator is the center of the actual profiler. It merges the eBPF code from all the loaded modules, starts profiling using BCC, manages PIDs to track (more on that below), and finally sends the collected traces to TraceHub.

What PIDs are being tracked?
Initially, no PID (Process ID) is tracked and therefore no tracing data is gathered. Upon `execve` system call, the user-space program is notified and checks whether the newly started process has the environment variable `ENABLE_IO_PROFILING` set to 1.

If that is the case, the PID of the newly started process is added to an eBPF-HashMap, called `tracked_pids` and from that moment on, tracing data will be gathered for this process and every other process contained in this HashMap.

Figure 4.4 display this process visually. The yellow box shows an external event, the process can e.g. be started by the user manually or by another parent process. The gray boxes all take place within `fsnoop` as it processes the start of a new process to determine whether operations of that PID should be tracked or not.

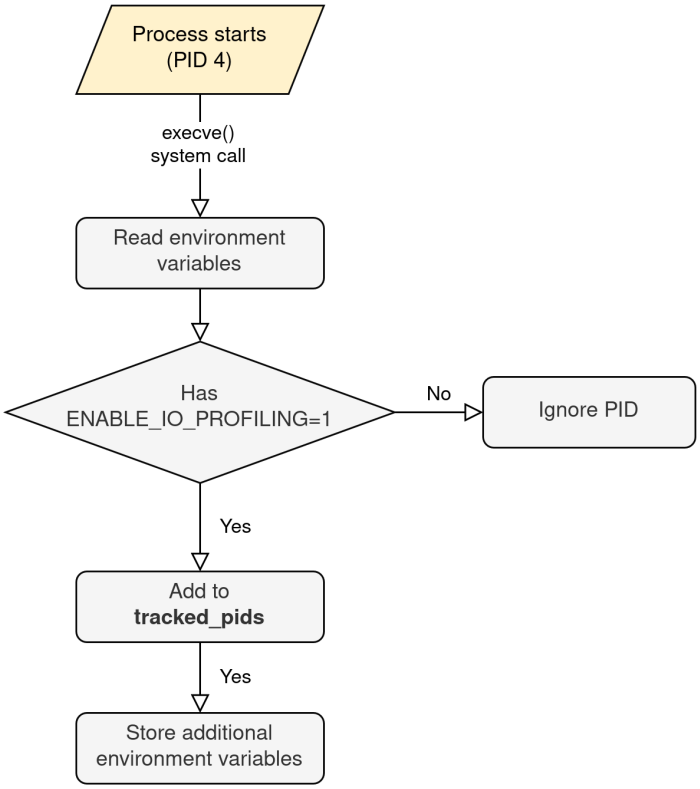


Figure 4.4: Simplified process of managing PIDs to track within `fsnoop`

The user-space program also tracks some additional environment variables set for the process, by reading them upon process startup. These variables are:

- Job-ID (default environment variable name is `PIPELINE_JOBID`)
  - can be overridden using the command-line argument `-env-job-id`
  - Job-ID can be seen as identifier for each executable in a pipeline
- Pipeline-ID (default environment variable name is `PIPELINE_PPOID`)
  - can be overridden using the command-line argument `-env-pipeline-id`
  - Pipeline-ID can be seen as identifier for a pipeline
- Pipeline-Path (default environment variable name is `PIPELINE_PIPELINEROOTPATH`)
  - can be overridden using the command-line argument `-env-pipeline-path`
  - Pipeline-Path is where the pipeline definition is located
- Trace-Identifier (environment variable is `TRACE_IDENTIFIER`, cannot be overridden)
  - will be used to identify traces collected in one run
  - can also be used to group traces of multiple pipelines with one identifier

4.2.6 PatternCoordinator

The PatternCoordinator is the counterpart to the CoreCoordinator, as it is being launched when data should not be gathered but rather retrieved and then analyzed. It first loads the data from TraceHub (according to the command-line arguments) and hands them to each PatternInterpreter specified in the configuration file for further analysis.

Parallelism

In order to have parallel execution of all interpreters, they are all started using a Python `ProcessPoolExecutor`.

4.2.7 PatternInterpreter

Each PatternInterpreter is responsible for analyzing the tracing data, processing it and thereby creating new outputs. These outputs are in the format of OpenTelemetry metrics and will be sent to the TraceHub. They can later further be analyzed in Grafana or other visualization platforms. Despite the name, PatternInterpreters do not really detect a pattern but rather provide metrics to detect patterns. This is necessary because not in every system, the same thresholds for those metrics would apply and therefore on some systems patterns might never be detected, while on others the interpreters would constantly trigger false-alarms. In the current solution, the following PatternInterpreters are implemented:

- overlapping (detects overlapping file accesses)
- lclu (measures cache line utilization)

Initially, a metadata_operations PatternInterpreter was also implemented, that should allow for easier detection of a high metadata synchronization overhead. However, this was rendered unnecessary as explained in section 4.7.2. Therefore this interpreter was removed from the source code.

Thanks to the flexible design with explicitly loading data on startup, the PatternInterpreters can first of all run retrospectively and don't have to run during the profiler execution, and secondly they can be repeatedly executed over and over again.

Another benefit of that design is that they are independent from the profiler and can be effectively executed on any node or any host as long as tracing data can be loaded and metrics sent from that host. This increases the overall performance and especially keeps overhead on the actual cluster nodes low since the PatternInterpreters do not have to be executed on those nodes.

Overlapping

The Overlapping Interpreter first groups traces only by their filenames, allowing it to check all tracked executables for any overlapping operations. The traces are filtered to ensure that no operation is interpreted that has not read any bytes. Afterwards, the traces are sorted by timestamp so that the chronological sequence of file accesses can be reconstructed. Only the `read`, and `write` operations are considered when searching for overlaps.

To efficiently and reliably detect these overlaps, the interpreter uses an interval tree. This data structure is optimized to detect overlapping intervals. As it processes each trace, when a file access begins, it adds the corresponding offset range (offset start to offset end) into the tree. Before inserting the new interval, it checks how many intervals are already present in that section of the tree. If none are found, no overlap has occurred. The number of overlaps is recorded in a metric along with the timestamp and offset start, allowing one to determine when and where a detected overlap occurred in the file [31] [32] [33].

Low cache line utilization

The Low Cache Line Utilization (lclu) interpreter organizes traces using several nested grouping levels. First, they are grouped by pipeline_id and job_id, then by filename, and finally by the type of operation. Only read- or cache-sensitive operations are tracked, i.e. `read`, `filemap_fault`, `mmap`, `do_mmap`, `handle_mm_fault`, and `munmap`.

In its initial phase, the interpreter focuses on measuring the difference in the offset between consecutive calls. The traces are sorted by timestamp so that the chronological sequence can be reconstructed. Then, the offset-difference between consecutive calls is measured and recorded as a metric. If many measurements show a high difference, it indicates that the memory cache

is not being utilized optimally, since this can indicate either random access or inefficient loop nesting rather than sequential reads/writes.

This measurement from above is then used to build context by calculating the average offset-difference (grouped still by pipeline_id, job_id, type, and operation). If the average difference is high, this indicates that the cache is not being used efficiently.

The configuration file of `fsnoop` allows the user to specify how many bytes are be prefetched by the file system during a read operation. This value is used to count how many operations occur within the prefetched parts of the file. A low count here suggests that file prefetching is underutilized. Finally, the interpreter combines the sequential count and the average difference into a single metric for graphical comparison.

4.2.8 Visualization

Data gathered both during tracing phase (by the profiler) as well as later on in the analyzation phase (by the PatternInterpreters) can be visualized in Grafana (or any other OpenTelemetry-capable visualization platform).

In the current state of the project, OpenTelementry-Collector is used to gather data and it then re-distributes it to Grafana-Tempo (in case of Traces) or Prometheus (in case of Metrics).

4.2.9 Templating

To ensure maximum performance and the smallest possible bundle as well as easier integration for newly developed Modules or PatternInterpreters, templating is used.

Effectively, the following files are all written as templates:

- `fsnoop_template` (results in final executable `fsnoop`)
- `core_coordinator_template` (results in `core_coordinator`)
- `pattern_coordinator_template` (results in `pattern_coordinator`)

Those templates already contain the whole logic required, but lack any imports of Modules or PatternInterpreters (as well as calls to them). Instead, they contain placeholders such as these:

```
# Module imports
%MODULE_IMPORTS%
...

def register_modules():
        ...
        %MODULE_REGISTRATIONS%
```

Listing 3: Example of placeholders within template-file `core_coordinator_template`

The placeholders in Listing 3 will then be replaced based on the configuration file. The configuration file might look as shown in Listing 4.

```
modules:
        - syscalls
        - mmap
        - vfs
patterns:
        - overlapping
        - lclu
...
```

Listing 4: Example configuration file for `fsnoop`

In this case, when the user executes the command below, the modules `syscalls`, `mmap`, and `vfs` will automatically be imported and their code will be registered and added to the final output. The same behavior occurs for the PatternInterpreters `overlapping` and `lclu`, that will be imported in `pattern_coordinator`.

```
// This command replaces all placeholders in the template-files
// based on the values in the configuration file
$ ./prepare
```

4.3 eBPF profiler

This section dives deeper into the implementation of the eBPF profiler within the tool `fsnoop`. It explains which hooks are chosen as well as why and how the probes are implemented (as briefly outlined in section 4.2.1).

4.3.1 Hook types

In general `kprobes` and `kretprobes` are used in `fsnoop`, since they allow capturing both the entry as well as the exit of a kernel function. They can hook into system calls but also any other kernel-internal function. This allows for maximum flexibility and even though the `kprobe`-API is less stable than the `tracepoints`-API, the file system operation API did not really change in the past few kernel versions (e.g. kernel version 5.14 and kernel version 6.11 are compatible in all those cases).

4.3.2 eBPF hooks chosen

The following hooks have been implemented in each Module using eBPF to gather the data needed for interpreting file access patterns.

Module `syscalls`

The following hooks are implemented in the module `syscalls`, which is meant to group all file-related system calls as described in section 4.2.3.

Type of hook	Function name	Purpose
kprobe	do_sys_openat2	register file open event
kretprobe	do_sys_openat2	save return value (file descriptor) along with file-name in file-descriptor map for lookup
kprobe	sys_close	register file close event, remove file from file-descriptor map
kprobe	ksys_read	register file read event
kretprobe	ksys_read	add duration and number of bytes read to file read event
kprobe	sys_write	register file write event
kretprobe	sys_write	add duration and number of bytes written to file write event
kprobe	sys_writev	register file writev event (write from multiple buffers [28])
kretprobe	sys_writev	add duration and number of bytes written to file writev event
kprobe	sys_pwrite64	register file pwrite event (write at specific offset [28])
kretprobe	sys_pwrite64	add duration, number of bytes written and calculated offset_end to file pwrite event
kprobe	sys_pwritev	register file pwritev event (write at specific offset from multiple buffers [28])
kretprobe	sys_pwritev	add duration, number of bytes written and calculated offset_end to file pwritev event
kprobe	sys_pwritev2	register file pwritev2 event (write at specific offset from multiple buffers [28], newer implementation)
kretprobe	sys_pwritev2	add duration, number of bytes written and calculated offset_end to file pwritev2 event
kprobe	do_sendfile	register sendfile event (efficiently transfer bytes from one file descriptor to another [28])
kretprobe	do_sendfile	add duration, number of bytes written and calculated offset_end to sendfile event
kprobe	do_sys_truncate	register file truncate event (set file to a specific length [28])
kretprobe	do_sys_truncate	add duration to file truncate event
kprobe	ksys_fallocate	register file fallocate event (allocate space on disk to make sure e.g. write won't fail [28])
kretprobe	ksys_fallocate	add duration to file fallocate event
kprobe	do_renameat2	register file rename event
kretprobe	do_renameat2	add duration to file rename event
kprobe	do_unlinkat	register file unlink event (remove link, delete file-name [28])
kretprobe	do_unlinkat	add duration to file unlink event

Table 4.1: eBPF probes implemented in `syscalls` module.

Module `vfs`

The following hooks are implemented in the module `vfs`, which is meant to group all relevant vfs calls as described in section 4.2.3.

Type of hook	Function name	Purpose
kprobe	vfs_open	register VFS file open event
kprobe	vfs_create	register VFS file create event
kprobe	vfs_read	register VFS file read event
kretprobe	vfs_read	add duration, number of bytes read and calculated offset_end to VFS file read event
kprobe	vfs_write	register VFS file write event
kretprobe	vfs_write	add duration, number of bytes read and calculated offset_end to VFS file write event
kprobe	vfs_iter_write	register VFS file iter-write event (write, possibly asynchronous, from multiple buffers [34])
kretprobe	vfs_iter_write	add duration, number of bytes read and calculated offset_end to VFS file iter-write event
kprobe	vfs_truncate	register VFS file truncate event (set file to a specific length)
kretprobe	vfs_truncate	add duration to VFS file truncate event
kprobe	vfs_fallocate	register VFS file fallocate event (preallocate blocks on disk to make sure e.g. write won't fail [34])
kretprobe	vfs_fallocate	add duration to VFS file fallocate event
kprobe	vfs_rename	register VFS file rename event
kretprobe	vfs_rename	add duration to VFS file rename event
kprobe	vfs_unlink	register VFS file unlink event (remove link, delete filename, called by unlink system call as in [28])

Table 4.2: eBPF probes implemented in `vfs` module.

Module `mmap`

The following hooks are implemented in the module `mmap`, which is meant to group all caching-related calls together as described in section 4.2.3.

Type of hook	Function name	Purpose
kprobe	sys_mmap	register mmap system call event (creates a new mapping in the calling process's virtual address space [28])
kretprobe	sys_mmap	add duration to mmap system call event
kprobe	do_mmap	register mmap kernel function call event (called by mmap system call)
kprobe	sys_munmap	register munmap system call event (removes an existing mapping from the calling process's virtual address space [28])
kretprobe	sys_munmap	add duration to munmap system call event
kprobe	exit_mmap	register exit_mmap event (unmaps all associated mappings, e.g. on exit of a process [35])
kretprobe	exit_mmap	add duration to exit_mmap event
kprobe	handle_mm_fault	register handle_mm_fault event (called an page fault, i.e. when page is not in RAM yet [35])
kretprobe	handle_mm_fault	add duration to handle_mm_fault event
kprobe	filemap_fault	register filemap_fault event (called an file-backed page fault, i.e. when page is not in RAM yet [35])
kretprobe	filemap_fault	add duration to filemap_fault event

Table 4.3: eBPF probes implemented in `mmap` module.

4.4 Data collection and storage

4.4.1 Telemetry types

Traces provide the necessary context to understand time sequences and relationships between different system components. In the case of `fsnoop`, a temporal context is established via the file system accesses. Each system or kernel call is stored as a trace entry. The probes gather data from multiple jobs and pipelines. Additional information is stored in the spans, so that data can be aggregated and correlations be identified. This provides an overview of the sequence of the pipelines being tracked. Environment variables determine which pipeline is traced and define the trace identifier [36].

4.4.2 Tempo

Tempo has been chosen as backend for storing tracing data. It was released by Grafana Labs in 2020 and focuses on easy-to-use, highly distributed, and efficient data storing. Since it was developed by Grafana Labs, it offers tight integration with Grafana and also has an integration for OpenTelemetry. For `fsnoop`, the OpenTelemetry Collector is used to send the collected data to Tempo. A trace identifier can be assigned to each measurement run for easier data categorization. In addition, the Tempo API is used to load the data for the interpreters. The API does not make it possible to load filtered traces with all spans directly, instead the filtered spans are loaded at first. In a second step, complete traces are then loaded using their respective IDs [37] [38].

4.4.3 Prometheus

Prometheus was published as an open source project in 2012. It is now considered the standard for storing metrics data. It is widely used in the Grafana community and has a strong integration with Grafana. As the project's requirements for metrics data are lower than for trace data, Prometheus fits well into the `fsnoop` concept. Since Prometheus is widely used, the OpenTelemetry Framework also offers an integration with Prometheus. However, this is associated with some limitations within the Python-SDK. More on this in section 4.6 [39] [40].

4.4.4 Dataobjects

Two data objects are used in `fsnoop`. The event is used for communication from the eBPF probes to the program running in user space. It contains the most important information provided by system calls and other kprobes. This event object is standardized so that all probes use the same object type. It is defined within the CoreCoordinator. Table 4.4 lists all the individual attributes and describes them briefly. When a new module for `fsnoop` is developed, it only needs to make sure to fill the event object correctly. The conversion into a trace object is handled by the CoreCoordinator.

Attribute name	**Type**	**Description**
pid	u32	the process id of the process executing the operation
comm	char[TCL*]	the name of the executable
fd	u64	the file descriptor (if available)
filename	char[128]	the name of the affected file
type	char[25]	the operation type
start_ts	u64	the timestamp when the operation was started
duration	u64	the duration of the operation
offset_start	u64	the start of the byte-range affected by the operation
offset_end	u64	the end of the byte-range affected by the operation
n_bytes	u64	the number of bytes affected by the operation
metadata_relevance	u8	the impact of this operation on metadata synchronization
stack_id	u64	the stack ID (used to locate the line of code; currently being ignored)

Table 4.4: Description of the event object

***** TCL = `TASK_COMM_LEN`, which is a constant in the Linux kernel that defines the maximum length of a task's (process/thread) name

The `fsnoop` trace object differs from a trace in Tempo. The `fsnoop` trace is the data object that contains all the information for a trace entry. It is very simple and consists of primitive data types, lacking any complex data structures. The attributes from the trace object are then used to fill the spans for Tempo, which enables the aggregation of data within Tempo based on various attributes. A list of `fsnoop` traces is passed to the pattern interpreters. The analysis, performed by the pattern interpreters, take place on this data object. Some of the information from the traces is reused in the metrics (in the form of labels), so that the metrics can be aggregated on the same basis. Table 4.5 lists and describes the individual attributes of the trace object, as well as the default values and their span names.

Attribute name	**Span name and default value**	**Description**
name	name ""	the trace identifier as defined in the environment variable TRACE_IDENTIFIER
job_id	job.id ""	the job ID as defined in the environment variable PIPELINE_JOBID
pipeline_id	pipeline.id ""	the pipeline id as defined in the environment variable PIPELINE_PPOID
pipeline_path	pipeline.path ""	the pipeline path as defined in the environment variable PIPELINE_PIPELINEROOTPATH
pid	process.pid -1	the PID of the process executing the operation
comm	process.executable.name ""	the name of the executable executing the operation
fd	- -1	the file descriptor of the affected file
filename	process.executable.path ""	the name of the affected file
type	probe.type ""	the type of operation
start_timestamp_ns	startTimeUnixNano -1	the timestamp when the operation was started in ns
duration_ns	probe.time.duration -1	the duration of the operation in ns
offset_start	probe.offset.start 0	the start of the byte-range affected by the operation
offset_end	probe.offset.end 0	the end of the byte-range affected by the operation
n_bytes	probe.offset.bytes 0	the number of bytes affected by the operation
metadata_relevance	probe.metadata.relevance 0	the impact of this operation on metadata synchronization

Table 4.5: Description of the `fsnoop` trace object

4.5 Performance and overhead

Since `fsnoop` is meant to run in an HPC-environment on multiple nodes, making it as performant as possible is crucial. Using eBPF already helps drastically in reducing the overhead generated by the profiler, but before being able to use it in production, measurements are required to verify performance.

4.5.1 Test environment

All tests were run on the same test system, locally on the notebook of Philippe Krüttli with the following specifications:

Hardware Specifications

- **Model:** HP ENVY x360 2-in-1 Laptop 15-ey0xxx
- **Processor:** AMD Ryzen 7 5825U with Radeon Graphics (8 cores, 16 threads)
- **Processor clock frequency:** 2 GHz base, 4.5 GHz boost
- **Processor architecture:** x86_64
- **Memory (RAM):** 32GB DDR4 @ 2667MHz
- **Storage:** 2TB Samsung SSD 970 EVO Plus
- **Graphics:** Integrated Graphics
- **Battery:** 51Wh

Software Specifications

- **Operating System:** EndeavourOS Linux (kernel 6.6.70-1-lts) / Windows 11 (Dual Boot)
- **Python Version:** Python 3.13.1
- **Benchmarking Tools:** Python, GNU time, iostat, sort, top, dd, fio

The notebook was plugged-in at all times and a full system-reboot took place before every new benchmark to ensure no measurements would influence each other.

4.5.2 Measurements

Read latency

This benchmark aims to simply measure the time it takes to read a sample test file filled with random data (generated with `dd if=/dev/urandom of=testfile bs=1M count=1024`) with and without `fsnoop` running in the background.

	Without fsnoop	With fsnoop	Overhead
real	0.352 s	0.385 s	9.38%
user	0 s	0 s	0.00%
sys	0.302 s	0.344 s	13.91%

Table 4.6: Impact of fsnoop on read latency in a lightweight task

The data presented in Table 4.6 is aggregated over 30 runs. It clearly indicates that for a computationally lightweight task such as this, an overall overhead of nearly 10% is quite significant.

File open and read

In this benchmark, the sample test file from 4.5.2 is opened using Python and the first 1024 Bytes of it are read into memory.

	Without fsnoop	With fsnoop	Overhead
Average time	0.237 ms	0.246 ms	3.79%

Table 4.7: Impact of fsnoop on file open and read speed

The data presented in Table 4.7 is aggregated over 30 runs. An overhead of about 3.79% is definitely acceptable although the proportionally big overhead can be explained, since the time spent actually doing the task is extremely short.

CPU times

This benchmark simply shows the percentage of CPU time spent in user space vs. kernel space and in idle. The data is retrieved using: `top -b -n1 | grep "Cpu(s)"`

	Without fsnoop	With fsnoop	Overhead
User (user space)	1.72%	0.89%	-51.74%
System (kernel space)	1.52%	1.60%	**5.26%**
Idle	96.42%	97.16%	0.77%

Table 4.8: Impact of fsnoop on CPU-time spent in Kernel-space

The data presented in Table 4.8 is aggregated over 30 runs. The only relevant metric here is System time, as the profiler itself (the probes) run in kernel space. The execution of `fsnoop` increases the time spent in kernel space by 5.26%, however, one must consider that the system itself was left at idle during that time.

I/O wait

This benchmark shows again a statistics of CPU time, however, this time focusing on time spent waiting for I/O requests (I/O Wait) and time spent on kernel tasks (System). The data is retrieved using: `iostat -x 1 30`

	Without fsnoop	With fsnoop	Overhead
I/O Wait	0.028%	0.036%	28.57%
System	1.180%	1.443%	22.29%

Table 4.9: Impact of fsnoop on I/O wait times

The data presented in Table 4.9 is aggregated over 30 runs. It seems like the overhead is huge and extremely relevant. However, since the idle system had basically no I/O wait times to begin with, going to 0.04%, while being a high relative increase, still shows that even with `fsnoop` running in the background, there is basically no I/O Wait happening.

Simulated workload

This benchmark tries to somewhat simulate a real-world scenario by reading a file's content, sorting it using the `sort` command in Linux and then writing the sorted content into another file.

	Without fsnoop	With fsnoop	Overhead
real	5.127 s	6.023 s	17.48%
user	19.870 s	21.736 s	9.39%
sys	0.709 s	1.267 s	78.70%

Table 4.10: Impact of fsnoop on a simple simulated workload (file content sorting)

The data presented in Table 4.10 is aggregated over 30 runs. It shows a rather high overhead generated by fsnoop. It is not surprising, that the amount of time spent in kernel space is much higher, but an overall overhead of 17.48% is very high. That is why the project team decided on running some other simulated workloads to compare results (see below).

Sequential disk read/write (dd)

This benchmark tries to simulate general file system operation by triggering sequential reads and writes using `dd`.

	Without fsnoop	With fsnoop	Overhead
Throughput	2.182 GB/s	2.203 GB/s	-0.96%*
Duration	0.527 s	0.525 s	-0.38%*

Table 4.11: Impact of fsnoop on sequential reads and writes (*negative Overhead explained below)

The data presented in Table 4.11 is aggregated over 30 runs. It shows a negative overhead, meaning the system performed even better with `fsnoop` running than without. However, since the margin between the measurements is so small it can be assumed that `fsnoop` just has no overhead on sequential reads and writes generated by `dd`.

Pipeline simulation (fio)

This benchmark as the final one tries to simulate the behavior of pipelines as they appear in HPC-environments. Using `fio`, two workers are generated that construct files and write content into them and two workers are generated that consume this data by reading the generated files. This somewhat mimics the behavior we can expect when one executable in the pipeline writes an output file, that is then used by the subsequent executable.

	Without fsnoop	With fsnoop	Overhead
Read bandwidth	1,095 MiB/s	1,071 MiB/s	2.19%
Read IOPS	8,762	8,566	2.24%
Read latency	199.6 μs	205.2 μs	2.81%
Read disk utilization	78.45%	76.08%	3.02%*
Write bandwidth	2,544 MiB/s	2,442 MiB/s	4.01%
Write IOPS	20,350	19,540	3.98%
Write latency	97.3 μs	101.9 μs	4.73%
Write disk utilization	71.73%	69.13%	3.62%*

Table 4.12: Impact of fsnoop on a pipeline-like execution flow (*disk utilization is only really relevant when above 90% indicating a bottleneck)

The data presented in Table 4.12 is aggregated over 10 runs. It shows an overhead of about 2.2-2.8% on read operations in bandwidth, IOPS and latency. The overhead on write operations seems a bit larger with about 4-4.7% in bandwidth, IOPS and latency. These measurements also seem to be coherent with about the same overhead in all read metrics and write metrics respectively.

4.5.3 Interpretation and conclusion

While many benchmarks provided different results, in general the overhead seems to be especially high when the underlying operation is extremely lightweight. The heavier the operations and the closer the benchmarks try to emulate a real-world scenario, the closer we get to both an acceptable as well as a realistic result.

About **2-2.8% overhead** on read operations and **4-4.7% overhead** on write operations, resulting in about **3.4% overhead**, in a scenario where read and write operations are balanced, is very acceptable.

However, since most pipeline-architectures have much more read operations with multiple executables reading the same files while only writing one or a few output files, a realistic overhead in HPC-environments can be expected aroud **2.5-3%** overall.

Scaling

Since `fsnoop` will run on every worker node, the overhead on the cluster will remain the same, regardless of the number of worker nodes added or removed. It is expected that with a growing number of CPU cores and more system resource utilization, the overhead might even be reduced as the measurements above have been made on a regular notebook with extremely limited resources compared to computers in HPC environments. In those environments, the base overhead generated by just having `fsnoop` running in the background will be negligible and the only real overhead will be the one generated by the actual probes.

Comparison

In general, `fsnoop` is well within range of other eBPF-based tools or profilers in terms of overhead generation. An anomaly detection system, developed in 2018 at TU Berlin for example generates around 3% CPU overhead on a 10GB/s network monitoring [41]. Flora, a cloud-native eBPF-based monitoring tool to gather data on network traffic, application performance, and system resources generates about 9% CPU overhead [42]. In 2024, six students at Virginia Tech compared the overhead of pre-, in- and post-eBPF filtering of processes. Their best result used pre-eBPF filtering in which they induced 15% overhead on the `read` system call and 6% overhead on the `sendmsg` system call [43].

Even Darshan, which is considered an industry standard for I/O profiling remains at about 2% overhead on I/O times, which is comparable with our results [44].

4.6 Challenges and limitations

4.6.1 Project planning

The biggest challenge in this bachelor's thesis was the extremely tight schedule. The project's scope was underestimated from the beginning, leading to a short and intense implementation phase. This is significant because many subsequent decisions were driven by these scope limitations. The focus was placed on the core functionality of `fsnoop`, optimizations and additional functionalities were prepared or discussed theoretically in chapter 5.3 Future work. In retrospect, splitting the project into a theoretical, conceptual part and a separate implementation thesis would have enhanced the overall quality. However, this realization came too late and consequently the work was confined to the essential functionality. The system was, however, built with this limitation in mind, therefore allowing for easier future extensions. The primary goal of developing a deployable application has been achieved, and now it is time to build upon that foundation.

4.6.2 Deployment on DEV-Cluster

This section outlines the challenges encountered during the deployment on an HPC environment, as similar issues may arise in future deployments.

Through close collaboration with the Royal Observatory, Edinburgh (ROE), `fsnoop` was deployed on a real HPC cluster to profile a test pipeline in a realistic setting. This deployment would not have been possible without their support, as the profiling step requires root-privileges, which are also necessary for installing dependencies.

The setup script in the `fsnoop` repository is designed to install all required dependencies and was modified to use the `yum` package manager, as this is the package manager used by the ROE DEV-Cluster. The first challenge involved the network infrastructure: the compute nodes do not have direct internet access, all outbound traffic must pass through a proxy. To address this, the proxy configuration was incorporated into the setup script, enabling a successful start of `fsnoop`.

Once the test pipeline was initiated, another issue became apparent. The `fsnoop` profiler writes log statements to the console (`STDOUT`) rather than log files. Initially, no data appeared in Tempo, making it necessary to inspect the process output. To facilitate this, `fsnoop` was restarted with output redirection to a log file using the following command:

```
// Start fsnoop and redirect output to logfile
$ ./fsnoop &> fsnoop.log
```

With the log files available, it was confirmed that `fsnoop` was running correctly without errors. However, an additional issue was identified: the infrastructure components, including Tempo and the OpenTelemery Collector, were misconfigured on the cluster. Specifically, the OpenTelementry Collector was not yet set up to send data to Tempo, and its batch size (the number of data points sent per batch) needed to be reduced. After adjusting these configurations, data successfully arrived in Tempo, completing the deployment process.

4.6.3 Timestamping limitations

The OpenTelemetry SDK uses the MetricReader to define when the collected data is sent to Prometheus. In practice, the only concrete implementation provided is the PeriodicExportingMetricReader. Since this is the standard option documented by OpenTelemetry, we decided to use it instead of developing our own reader. The principle is quite simple. The metrics are collected by the system and gathered in-memory at the OpenTelemetry MetricExporter. After the defined time interval has elapsed, the collected data is pushed to Prometheus. If the metrics share identical label values, they are aggregated into a single metric.

This approach only makes sense if the data is collected at runtime. However, in `fsnoop` the interpretation of the data has been outsourced to the interpreters. This allows for the traces to be interpreted at a later point in time. But as a result, the interpreters require the ability to set a metric's timestamp manually. The PeriodicExportingMetricReader does not allow for that, which is a significant limitation imposed by the current OpenTelemetry SDK.

To work around this, the original timestamp is stored within the metric labels. In Grafana, one can then convert these label values into numerical timestamps and display the data in an X/Y-chart with the X-axis representing the time. This simple workaround saves development time but is not an ideal solution. In order to completely solve the challenge in the future, one of the following workarounds would have to be used.

- Adding a new processor in the OpenTelemetry Collector that transfers the timestamp from the labels into the actual time series timestamp.
  - This solution remains theoretical and has not been practically proven to be allowed by OpenTelemetry.
- Create a custom ExportingMetricReader that does not set the timestamp automatically but instead allows it to be set manually and pushes a metric entry directly.
  - In other words, this would change the focus away from live measurements to a subsequent creation of metrics as in `fsnoop`.
  - However, this approach may face hurdles due to inherent limitations in the SDK.
- Bypass OpenTelemetry completely and build a custom metric collection system
  - This would offer full flexibility, but is considerably more labor-intensive
  - One would no longer benefit from using a proven framework

[45] [46]

4.6.4 File system filtering

To enhance the filtering of trace data, one approach was to determine from which mounted file system the data was loaded. This would allow filtering traces so that only those from a specific mounted file system are traced. In an earlier version, `fsnoop` retrieved the mount points of the system. Subsequently, an identifier should be captured during the path lookup process, which then enables filtering in the probes. This function was removed from `fsnoop`, however, because no suitable identifier could be defined in the path lookup. The goal was to have an identifier that could be used to filter using a simple comparison. However, a basic string comparison on the path itself is insufficient. With further, more detailed analysis of the path lookup process, this functionality might later be reintegrated into the system. [47]

4.6.5 Line of code

To provide users with more precise information about which executables exhibit inefficient access patterns, the analysis would need to be broken down to the level of methods or individual lines of code. This approach was investigated using a prototype. Breaking down the stack to the exact Python source code lines rather than tracking the Python interpreter brings up several challenges. Profiling interpreted code is challenging because it requires a deep understanding of the runtime's internals, particularly the runtime itself and the abstract stacks that track variables and control flow. In essence, effective profiling involves identifying the data structures that hold the virtual machine's state and reconstructing complete stack traces to capture how high-level code is processed [47].

Because fully resolving the stack is not considered a core feature, this functionality was removed from `fsnoop`. It was, however, prepared such that all probes already collect the stack ID. These IDs are then currently being ignored when converting to traces to avoid confusion, since the stack ID would have to be fully resolved and just passing it as is would provide no benefit. However, this function still requires further analysis. One solution would be to introduce a new module that processes and resolves the stack ID before the traces are sent.

The prototype showed that essentially one could:

- Store the Stack ID
- Get the memory address associated with this Stack ID
- Subtract the base memory address of the executable from this address
  - then you end up with a relative address from the start of the executable
- resolve from relative memory address to line of code using tools such as `addr2line`
  - for Python code one cannot directly use this output but rather retrieve the number of the line of code associated with this memory address (which is part of the output)
  - then the source file could be parsed to retrieve the actual line of code

4.6.6 OS limitation

The goal was to ensure `fsnoop` runs reliably on the nodes in ROE's DEV-cluster, which run on the kernel version **5.14**. Development was conducted on Ubuntu 22.04.2 LTS with kernel version **6.11.0-19-generic** as well as on EndeavourOS with kernel version **6.6.70-1-lts**, ensuring that `fsnoop` is fully functional on newer and more modern systems as well. In theory, `fsnoop` should also run on older kernel versions since the probes are mostly based on system calls that rarely change. The kprobes used originate from functions that have been in use for a long time and are rarely modified. The implementation was further improved by using a more abstract system call method. However, `fsnoop` was only ever comprehensively tested and verified on the abovementioned versions.

4.6.7 Data processing

The data processing is limited by Tempo. During the loading process of the data, the time range specified must not exceed 168h. Additionally, when data loading from Tempo takes too long, a timeout is triggered. Tempo is also limited in the number of data points per query. This limitation depends on the system on which `fsnoop` or Tempo is running. Through optimizations in the settings, we were able to reliably load up to 200,000 records, although our test collected a total of 11,960,219 trace data points. This indicates that, at current state, it is not possible to analyze the entire dataset in one run.

Primarily, this limitation can be addressed through targeted optimizations in the Tempo configuration. Currently, Memcached is used for caching since it is provided as the default implementation. The settings in Memcached might also offer potential for further performance improvements. It is also advisable to plan the tracing based on the number of measurement data points, rather than collect data of multiple pipelines with the same trace identifier. These limitations apply especially to the data interpretation by the PatternInterpreters, meaning that sensible data aggregations should be chosen, so that only a portion of the entire data is interpreted at any one time. Apart from the interpreters, Grafana might also have its own limitations, however in our tests none were noticeable [48] [49] [50].

4.7 Summary

This chapter introduced the implementation `fsnoop`, a file access profiler that stands out thanks to its high modularity and flexibility. It efficiently gathers data on file accesses performed by various executables in an HPC environment, which can later be used to identify inefficient file operations. Additionally, `fsnoop` includes interpreters that analyze the collected data. It operates with widely used standard tools such as Prometheus, Tempo, and Grafana, which means there are no steep learning curves, making the use of `fsnoop` extremely straightforward for an experienced user in the HPC environment with no additional training required. If the scope of `fsnoop` is insufficient for any use case, it can easily be extended. If additional system calls, kprobes, or even new uprobes are needed, they can be developed as separate modules and easily integrated into the fsnoop configuration. These modules are subsequently loaded and deployed by the core coordinator.

Due to the use of eBPF and the fact that they contain no business logic, the probes are exceptionally powerful and generate minimal overhead, measured at only $\sim$2.5-3%. Startup parameters and the core coordinator ensure that only the necessary probes are loaded, preventing any unnecessary code from entering the kernel. After successful data collection, the pattern interpreters take fsnoop to a whole new level by adding valuable context to the collected data. These interpreters can also easily be extended through configuration, and users can select which ones to activate during runtime to optimize performance. These interpreters are completely decoupled from data collection, which allows for them to be executed on other nodes or even at a later time.

There is no comparable profiler, that captures file accesses with such detailed granularity without causing significant overhead, or influencing the executed code, on the market. This unique application lays the foundation for groundbreaking innovations and opens up exciting opportunities to fully exploit the hitherto untapped potential in performance analysis.

4.7.1 Results

Even the best-designed software is useless if it cannot provide any real value to its users. Therefore, this section presents an analysis of the results, and the data gathered and interpreted by `fsnoop` during a test run.

Thanks to the help of the Royal Observatory, Edinburgh (ROE), `fsnoop` could be tested on a real HPC cluster that is part of Euclid's cluster infrastructure. This test run included a pipeline called `MER_ProcessTile_EUCLID_2.0.2`, which included 34 different executables, processing 162 input files in total and producing 62 output files. The profiler of `fsnoop` was started beforehand on every node, that would run any executable of this pipeline.

During the full test run, which lasted from 13 March, 2025 17:52 UTC to 22:23 UTC, a total of **11,960,219 traces** were collected. Only a subset of those, however, was used in the visualizations to extract key performance indicators (see section 4.6.7 for more information on why only a subset of data could be used).

Figure 4.5 shows, that the `read` operation was the most frequently executed operation during this trace run. With the `handle_mm_fault`, however, one has to consider that it logs every mmap fault, including those instances when memory is allocated for the program in RAM. This system call should therefore always be considered in relation with `filemap_fault`, since `filemap_fault` actually loads the pages of the file from the file system into RAM. Notably, only 17 instances of data being directly loaded from the file system were recorded. Given the large number of read operations, this suggests that files were mostly read sequentially. Furthermore, the graphic shows that the entire RAM was cleared 39 times, which happens e.g. on the start of an executable.

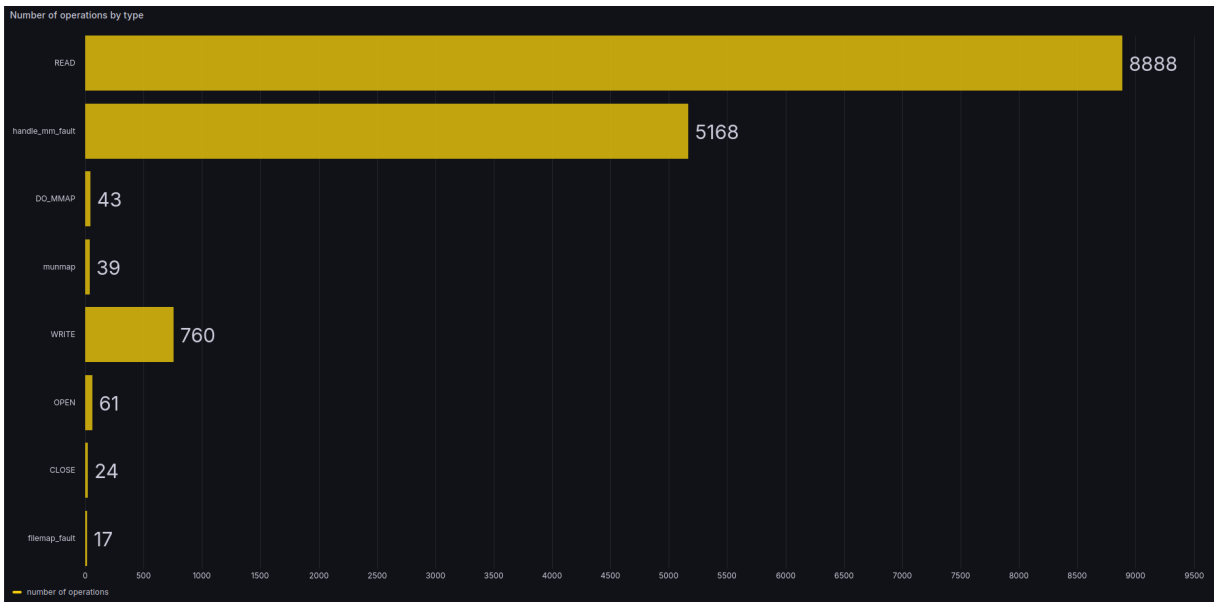


Figure 4.5: An overview of traces collected in the Euclid test run, split by operation type

The overview in Figure 4.6 shows how many operations were caused in total by which executables.

Source of traces by executable

Executable (source of trace)	Number of traces	Percentage of traces
python3	11225	62.4%
sourcextractor+	1954	10.9%
WsBackgroundMai	895	4.97%
sex	599	3.33%
CT_SWarp	3314	18.4%
unshare	3	0.0167%
bash	9	0.0500%
MER_Initialize	1	0.00556%

Figure 4.6: An overview of how many operations were caused by which executable during the Euclid test run

The visualization in Figure 4.7 shows the average duration of each file operation. It shows that `filemap_fault` had a very short average duration, indicating efficient loading from the file system. It is also apparent that the `read` calls were quite fast, which suggests that they were not blocked by e.g. the `write` calls. This is extremely important, considering how many `read` calls there were. From that fact we can conclude, that possibly no overlapping file access occurred.

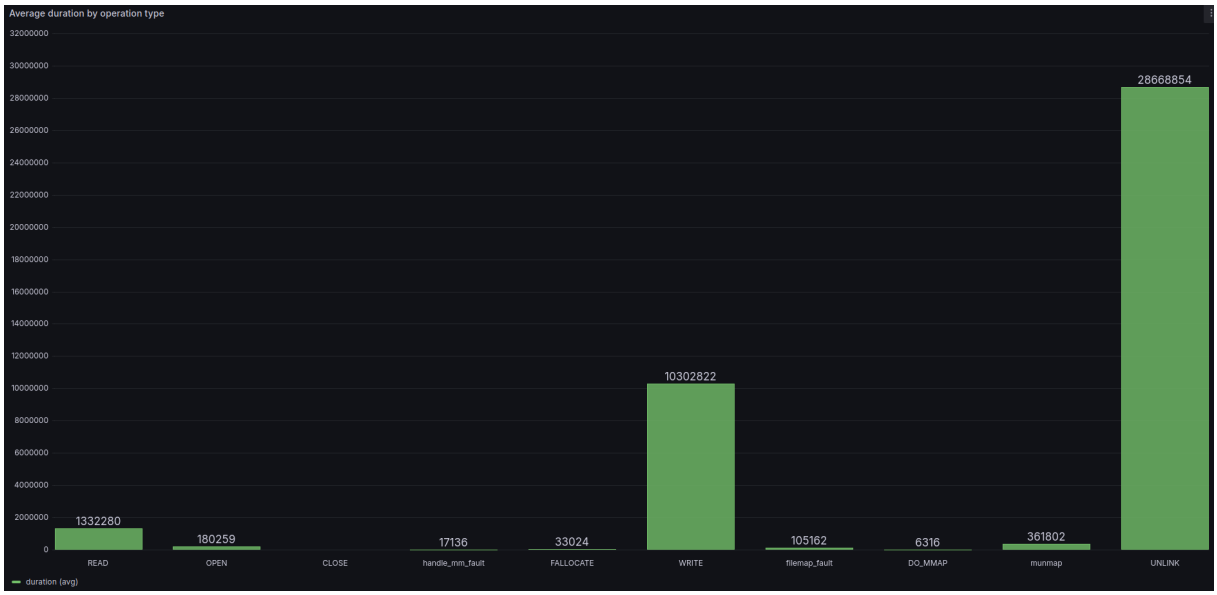


Figure 4.7: The average duration of each operation type in the Euclid test run

Figure 4.8 illustrates the duration of spans over time. One can clearly identify that between 20:00 UTC+1 and 20:10 UTC+1 there was a peak with apparently at least some long-running operations. And indeed, as the table in Figure 4.9 shows, there were some extremely long-runing operations (`read` and `write` operations taking over 1 minute) during that time.

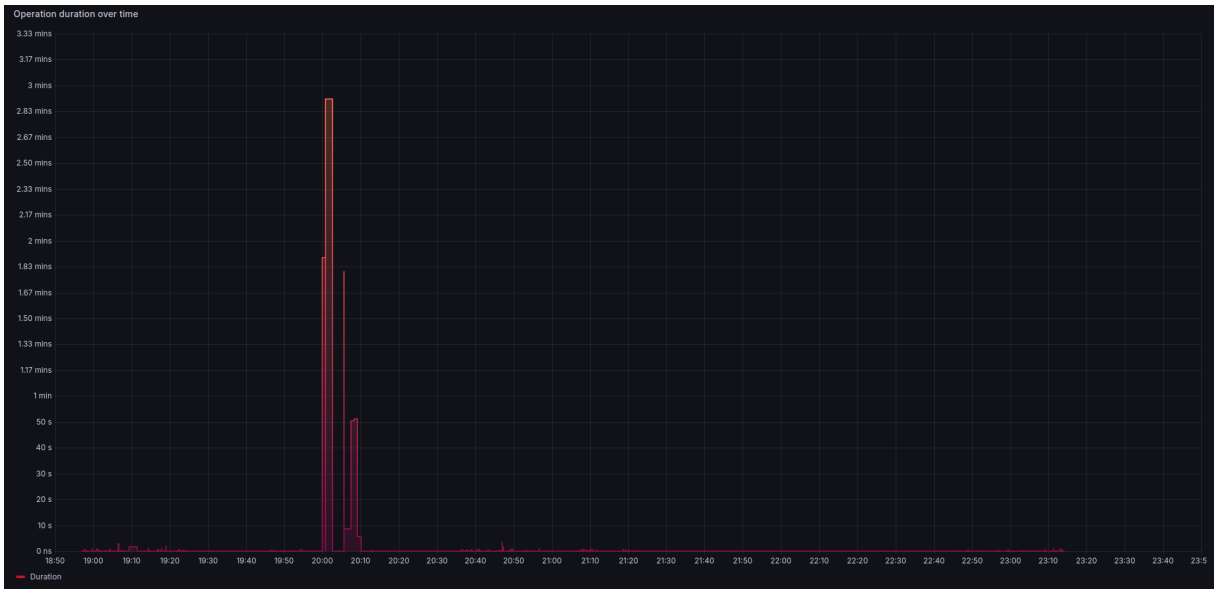


Figure 4.8: The average span duration over time in the Euclid test run

Long-running operations (>1m)

Trace Name	Span ID	Start time	Job	Pipeline	Operation	File	Duration
EUCLID-TESTRUN-2.0	973fe90f5a483388	2025-03-13 20:05:35	MER_ProcessTile_EUCLID_2.0	MER_ProcessTile_EUCLID_2.0	WRITE		1.80 mins
EUCLID-TESTRUN-2.0	6306da7615d2ebf9	2025-03-13 20:05:35	MER_ProcessTile_EUCLID_2.0	MER_ProcessTile_EUCLID_2.0	READ		1.80 mins
EUCLID-TESTRUN-2.0	f9c762d9f60304aa	2025-03-13 20:02:40	MER_ProcessTile_EUCLID_2.0	MER_ProcessTile_EUCLID_2.0	WRITE		2.91 mins
EUCLID-TESTRUN-2.0	2ad9053c908fd4bd	2025-03-13 20:02:40	MER_ProcessTile_EUCLID_2.0	MER_ProcessTile_EUCLID_2.0	READ		2.91 mins
EUCLID-TESTRUN-2.0	7a9a303d0ae60d50	2025-03-13 20:00:46	MER_ProcessTile_EUCLID_2.0	MER_ProcessTile_EUCLID_2.0	WRITE		1.89 mins
EUCLID-TESTRUN-2.0	847a27b8a2fe9205	2025-03-13 20:00:46	MER_ProcessTile_EUCLID_2.0	MER_ProcessTile_EUCLID_2.0	READ		1.89 mins

Figure 4.9: A list of some long running operations in the Euclid test run, highlighting potential bottlenecks

Low cache utilization pattern

In the following, some visualizations for the interpretation of low cache line utilization are explained. There are three distinct visualizations, designed to help identifying suboptimal caching behavior. One visualization shows the evaluation of the real data. In addition, there is an evaluation with mock data to better explain the visualization. The visualizations display all data points across all executables and files. However, they could also be filtered by specific files, jobs, or access types. The access types are represented by colors with the following meaning:

- **Blue:** read
- **Yellow:** filemap_fault
- **Green:** handle_mm_fault
- **Red:** munmap

Position of the largest offset difference
In this visualization, the Y-axis displays the actual offset, while the X-axis shows the offset difference to the previous operation. First, the distribution of data points helps identifying hotspots areas, where a high number of read operations occur. This is clearly visible in the real data. The most read operations are performed between bytes 200,000,000 and 450,000,000, between bytes 900,000,000 and 1,100,000,000, and between bytes 1,200,000,000 and 1,400,000,000. Secondly, data points outside these hotspots tend to show a larger gap compared to the preceding operation. However, if points in the hotspot clusters are located in the upper part of the graph as well, this suggests that data is randomly accessed. In such cases, it might be beneficial to adjust the code to ensure data is read as sequentially as possible.

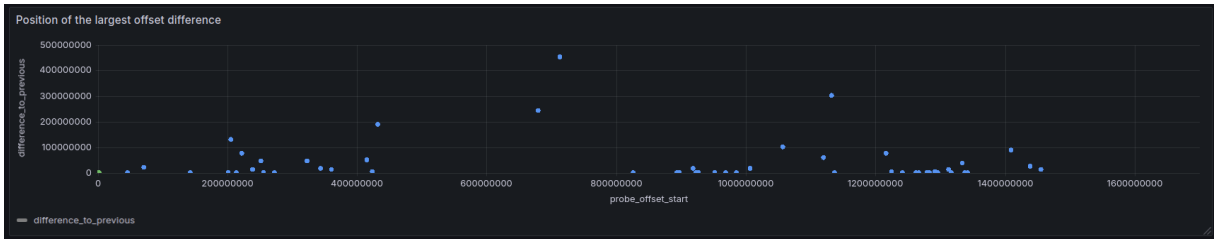


Figure 4.10: An overview of traces collected in the Euclid test run and interpreted with the lclu interpreter. Relationship differences to offset

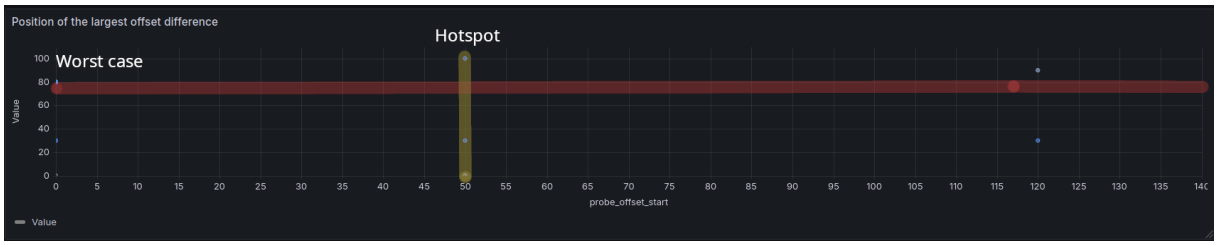


Figure 4.11: An overview of mock traces with described areas in diagramm. Relationship differences to offset

Ratio of Average Offset to Sequential
This visualization illustrates the relationship between the sequential count (number of sequential accesses) and the average difference of the offsets, which indicates whether prefetching is being utilized. The chart can be divided into four parts. The upper left represents the optimal case, where operations occur in quick succession and always fall within the prefetched segment of the file. In the bottom-left quadrant, the reads are sequential but prefetching is not used, suggesting that the prefetch settings could be refined. The bottom-right quadrant shows the worst case, where file accesses are likely random, meaning that the prefetched file segments are not used, thus cancelling the benefits of prefetching. The top-right quadrant, which should rarely occur, would indicate random accesses that, nonetheless, falls within the prefetched areas.

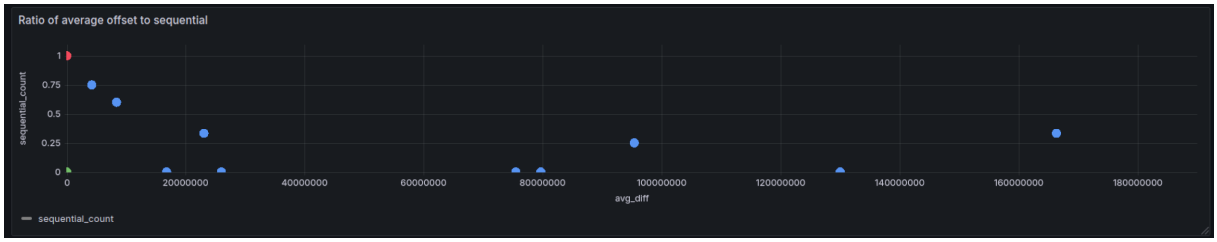


Figure 4.12: An overview of traces collected in the Euclid test run and interpreted with lclu interpreter. Relationship sequential count and difference

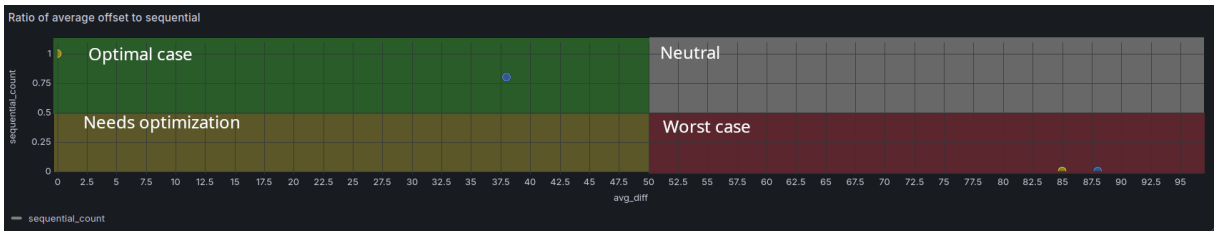


Figure 4.13: An overview of mock traces with described areas in diagramm. Relationship sequential count and difference

Average Offset Over Time

This visualization shows how the differences in offsets change over time, helping to determine at what point in time, the file is randomly accessed the most. One can also infer, at what time the file system was under the heaviest load. Different operation types can be linked together, which becomes apparent in the mock data. If a read call is executed and shortly thereafter a page fault occurs, these can be bundled together as a pair. When many such pairs occur throughout the file, this indicates that the RAM cache is not being used efficiently, resulting in the file being fetched from the file system more often.

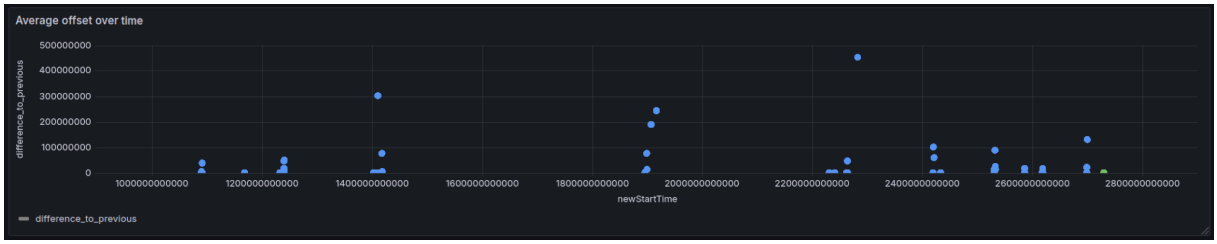


Figure 4.14: An overview of traces collected in the Euclid test run and interpreted with lclu interpreter . difference over time

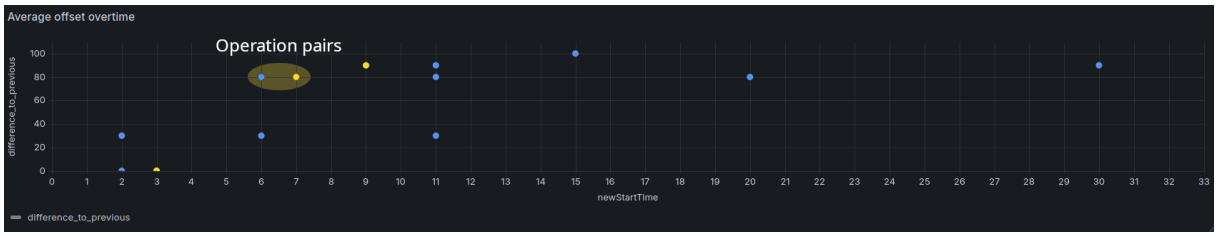


Figure 4.15: An overview of mock traces with described areas in diagramm. Difference over time

Overlapping pattern

This pattern uses the same real data as the low cache line utilization pattern, and it also includes a visualization using mock data, to clearly illustrate how overlaps are detected. The first visualization shows all files with the summed overlap value displayed for each file. The mock data clearly shows, that a total of six accesses to an overlapping part of the file occurred. The second visualization focuses on a single file's analysis. The Y-axis represents the start of the operation's offset, while the X-axis displays a numerical timestamp to visualize the sequence over time. Overlapping data points are highlighted in red, which is clearly visible in the mock data. As a nice side effect of this visualization is, that it can also be used to draw conclusions about lclu. The real data shows, that file accesses frequently occurred sequentially, with pairs of accesses being common.

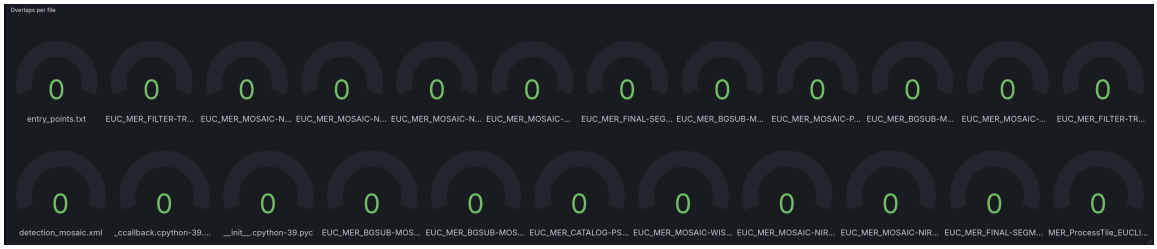


Figure 4.16: An overview of traces collected in the Euclid test run and interpreted with overlapping interpreter . Real data

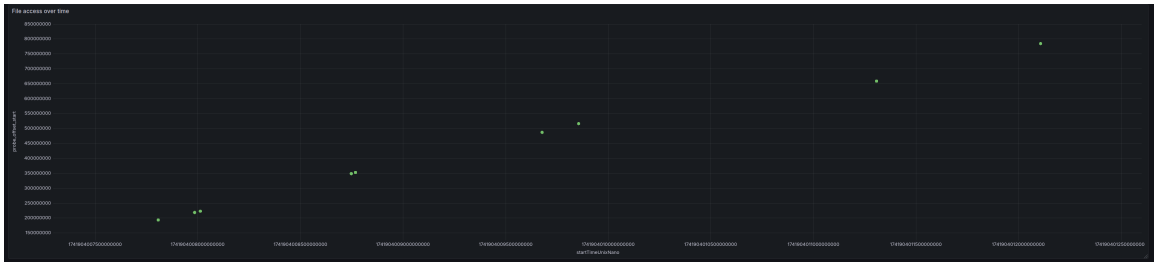


Figure 4.17: An diagramm of traces from one file collected in the Euclid test run and interpreted with overlapping interpreter

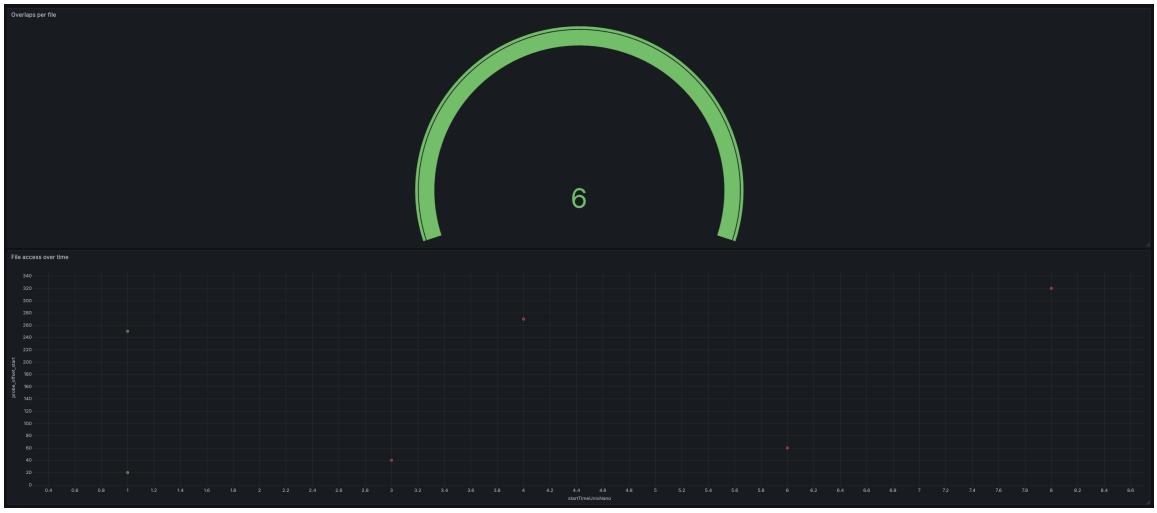


Figure 4.18: An overview of traces collected interpreted with overlapping interpreter . Mock data

4.7.2 Metadata synchronization overhead

While a metadata_operations PatternInterpreter, allowing for easier detection of high metadata synchronization overhead, was initially planned and also developed, results from just interpreting the traces showed that there was no added value by such an interpreter. All the necessary evaluations and visualizations could already be made using the traces.



Figure 4.19: The change in summed up metadata synchronization overhead (as calculated in attribute `metadata_relevance` of a trace) over time

For example, Figure 4.19 shows how much total `metadata_relevance` there was at any point in time. The value `metadata_relevance` measures the impact of a file system operation on metadata synchronization. It started high, indicating many metadata-relevant operations, which also makes sense because usually in the beginning, there are many file open and file create operations. While the visualization above shows the total summed up across all traces over time, the same visualizations could also be made summed up by job_id or filename or any other data contained in the traces. An interpreter for metadata synchronization overhead would not have given any other results except for those values in the form of metrics instead of traces.

5 Conclusion and Future Work

5.1 Summary of Key Findings

High-Performance Computing (HPC) workloads often suffer from inefficient file access patterns. In those shared file systems, inefficient file access can lead to significant performance bottlenecks, potentially for other workloads running on the same cluster as well. Traditional profiling tools can introduce high overhead or they require the developer to include them at compile time. This thesis introduced `fsnoop`, en eBPF-based file access profiling tool designed specifically to monitor and analyze file access patterns efficiently and with minimal system impact.

The key contributions of this work include:

- The design and implementation of a modular and extensible profiling framework for file access
- Integration of the profiler with OpenTelemetry tools for real-time tracing and post-processing (visually and numerically)
- The selection of eBPF-based kernel probes to track file accesses and patterns with low CPU overhead (∼2.5-3%), making `fsnoop` viable for continuous use in production environments
- A performance evaluation demonstrating that `fsnoop` introduces minimal overhead, particularly in more realistic, pipeline-like HPC workloads
- A test run of the profiler on the DEV-Cluster of the Royal Observatory, Edinburgh (ROE) to confirm its viability in HPC environments
- A comparison with existing profiling tools (Darshan, IOscope, iotrace), highlighting `fsnoop`'s advantages in flexibility, real-time monitoring, and reduced dependency on specific software stacks

Despite these successes, the project encountered several challenges, including limited access to real HPC environments, scalability constraints, and timestamping issues with OpenTelemetry that required workarounds.

5.2 Discussion of fsnoop's Impact and Limitations

The development of `fsnoop` contributes to HPC performance optimization by providing deep insights into file access operations. The software's extensible architecture ensures that it can be adapted to different workloads and extended in the future with additional features. This makes `fsnoop` a valuable I/O profiling tool for HPC environments beyond the scope of ESA's Euclid mission.

However, `fsnoop` also has limitations:

- **Lack of large-scale testing:** Although benchmarks demonstrated `fsnoop`'s efficiency, all tests were conducted on a single notebook. A large-scale HPC evaluation is needed to confirm its performance under realistic workloads
- **No automated pattern detection:** Currently, `fsnoop` relies on manual interpretation of profiling data. While the pattern interpreters do analyze and process the gathered traces, real pattern recognition (potentially AI-driven) could help detect and suggest optimizations to the code
- **Limited MPI support:** Many HPC applications rely on collective I/O using MPI, which `fsnoop` does not currently analyze in depth

But these limitations also present opportunities for future improvements and projects.

5.3 Future Work

To further enhance the quality and output as well as broaden the use cases for `fsnoop`, several future research topics are proposed:

5.3.1 Scalability Testing on Large-Scale HPC Clusters

Deploy `fsnoop` on multiple real HPC clusters to evaluate its performance on different parallel file systems (Lustre, GPFS, BeeGFS). Analyze the impact of `fsnoop` on multi-node workloads with thousands of concurrent processes. Compare those results with existing profiling solutions such as Darshan, IOscope, iotrace or others.

5.3.2 Automatic Bottleneck Detection using AI/ML

Develop machine learning models to automatically detect inefficient file access patterns based on the already gathered data. Implement an optimization suggestion model to help scientists improve their code based on the pattern analysis. Train models on historical profiling data to predict performance bottlenecks. Compare specifically trained AI models with general-purpose pattern detection models.

5.3.3 Enhancing Metadata Profiling & Cache Analysis

Extend fsnoop to profile how much time is spent synchronizing metadata vs. actual file I/O. Attempt to correlate metadata spikes with performance degradation. Improve cache analysis by tracking not only cache misses but also explicit inefficient prefetching, and poor memory locality.

Addressing these areas will help `fsnoop` evolve into a fully autonomous and scalable profiling solution for HPC environments.

5.4 Final Remarks

The optimization of file access in HPC environments is a crucial but often overlooked challenge. This thesis demonstrated that eBPF-based profiling can provide valuable insights with minimal overhead, making it a practical solution for large-scale workloads.

While `fsnoop` in its current form is already useful, it is specifically designed to be extended in the future. Further development is necessary to enhance its scalability, automate bottleneck detection and enhance analysis capabilities. By continuing this work, `fsnoop` could become an essential component of HPC performance optimization, helping researchers and engineers maximize computational efficiency and thereby contributing to the sustainability of research.

Future contributions to this project will be crucial in refining `fsnoop` into a state-of-the-art file access profiling tool for any modern HPC system.

6 Usage of external tools and AI

This thesis has been fully created by the authors themselves. The tools ChatGPT and DeepL were used selectively but across the report for paraphrasing.

Bibliography

[1] F. Nordwestschweiz. „«wir sind von a bis z verantwortlich»“. (2024), [Online]. Available: https://www.fhnw.ch/de/medien/newsroom/news/wir-sind-von-a-bis-z-verantwortlich (visited on Oct. 25, 2024).

[2] ESA. „Esa - euclid overview“. (Sep. 17, 2024), [Online]. Available: https://www.esa.int/Science_Exploration/Space_Science/Euclid_overview (visited on Sep. 17, 2024).

[3] L. Rice, *Learning eBPF*. O'Reilly Media, 2021, ISBN: 9781098135119. [Online]. Available: https://learning.oreilly.com/library/view/learning-ebpf/9781098135119/.

[4] W. Cadena, A. Ng, and M. Phillips, *Bcc tools - info poster*, https://www.lanl.gov/org/ddste/aldsc/hpc/_assets/docs/posters/BCC-tools-Info-poster.pdf, Mentors: Travis Cotton, Nick Jones, Johnathan Nielsen. Accessed: 24-Sep-2024, Los Alamos National Laboratory, 2024.

[5] G. Johar and V. Marupadi, *Bringing ebpf and cilium to google kubernetes engine*, https://cloud.google.com/blog/products/containers-kubernetes/bringing-ebpf-and-cilium-to-google-kubernetes-engine?hl=en, Accessed: 24-Sep-2024, Google Cloud, 2024.

[6] A. SkyWalking, *Profiling concepts and designs*, https://skywalking.apache.org/docs/main/next/en/concepts-and-designs/profiling/, Accessed: 2024-11-24, 2024.

[7] N. Kumar, *Monitoring applications with apache skywalking*, https://naveenrk22.medium.com/monitoring-applications-with-apache-skywalking-dc2e3e9f63b1, Accessed: 2024-11-24, 2023.

[8] T. Team, *What is apache skywalking?*, https://tetrate.io/blog/what-is-apache-skywalking/, Accessed: 2024-11-24, 2023.

[9] I. Gunaratne, *Integrate istio and apache skywalking for kubernetes observability*, https://imesh.ai/blog/integrate-istio-and-apache-skywalking-for-kubernetes-observability/, Accessed: 2024-11-24, 2023.

[10] A. N. Laboratory, *Darshan runtime: Installation and usage guide*, Accessed: 2024-11-24, 2024.

[11] S. Snyder, P. Carns, K. Harms, R. Ross, G. K. Lockwood, and N. J. Wright, „Modular hpc i/o characterization with darshan“, Argonne National Laboratory, Tech. Rep., 2016, Supported by U.S. Department of Energy, Office of Science. [Online]. Available: https://www.mcs.anl.gov/papers/P6048-0916.pdf.

[12] NERSC, *Darshan i/o profiler documentation*, https://docs.nersc.gov/tools/performance/darshan/, Accessed: 2024-11-24, 2024.

[13] N. A. S. Division, *Using darshan for i/o characterization*, https://www.nas.nasa.gov/hecc/support/kb/using-darshan-for-io-characterization_681.html, Accessed: 2024-11-24, 2024.

[14] G. K. Lockwood, *Building and managing darshan*, https://www.glennklockwood.com/hpc-howtos/managing-darshan.html, Accessed: 2024-11-24, 2023.

[15] C. Xu, S. Snyder, O. Kulkarni, *et al.*, „Dxt: Darshan extended tracing“, in *Proceedings of the HPC Conference*, Detailed evaluation included case studies on Cori and Blue Waters systems. Contact U.S. Department of Energy for further inquiries., IEEE or Relevant Publisher (confirm with actual conference details), 2020. [Online]. Available: https://cug.org/proceedings/cug2017_proceedings/includes/files/pap105s2-file1.pdf.

[16] A. Saif, L. Nussbaum, and Y.-Q. Song, „Ioscope: A flexible i/o tracer for workloads' i/o pattern characterization“, in *ISC High Performance 2018 International Workshops - WOPSSS'18*, Springer, 2018, pp. 57–70. DOI: 10.1007/978-3-030-02465-9_5. [Online]. Available: https://inria.hal.science/hal-01828249/file/IOscope_ISC2018_WOPSSS.pdf.

[17] LeUnAiDeS, *Ioscope toolkit*, https://github.com/LeUnAiDeS/IOscope, Accessed: 2024-11-24, 2024.

[18] O. CAS, *Standalone linux io tracer*, https://github.com/Open-CAS/standalone-linux-io-tracer, Accessed: 2024-11-24, 2024.

[19] O. CAS, *Open cas linux admin guide: Introduction*, https://open-cas.com/guide_introduction.html, Accessed: 2024-11-24, 2024.

[20] I. Corporation, *Intel® standalone linux io tracer user guide*, Revision 002, Document Number: 343366-002US, Accessed: 2024-11-24, 2020.

[21] I. Hu, „Measuring file access patterns in unix“, *SIGMETRICS Perform. Evaluation Rev.*, vol. 14, pp. 15–20, 1986. [Online]. Available: https://api.semanticscholar.org/CorpusID:11101030.

[22] Rogue Wave Software, Inc., *Freja manual*, Copyright © 2006-2016 Rogue Wave Software, Inc., ParaTools, Inc., 2836 Kincaid St., Eugene, OR 97405, 2016. [Online]. Available: http://www.nic.uoregon.edu/~khuck/ts/acumem-report/manual_html/index.html.

[23] T. Kroeger and D. Long, „The case for efficient file access pattern modeling“, May 1999.

[24] S. Thapaliya, P. Bangalore, J. Lofstead, K. Mohror, and A. Moody, „Io-cop: Managing concurrent accesses to shared parallel file system“, in *2014 43rd International Conference on Parallel Processing Workshops*, 2014, pp. 52–60. DOI: 10.1109/ICPPW.2014.20.

[25] CEA. „Tgcc - mpi-io“. (2024), [Online]. Available: https://www-hpc.cea.fr/tgcc-public/en/html/toc/fulldoc/IO.html (visited on Dec. 1, 2024).

[26] M. Forum. „Mpi 4.1 specification“. (2021), [Online]. Available: https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf (visited on Dec. 1, 2024).

[27] R. Latham. „Introduction to mpi-io“. ATPESC 2023. (2023), [Online]. Available: https://extremecomputingtraining.anl.gov/wp-content/uploads/sites/96/2023/08/ATPESC-2023-Track-7-Talk-7-latham-mpiio.pdf (visited on Dec. 1, 2024).

[28] M. Kerrisk, *The Linux Programming Interface*. San Francisco, CA: No Starch Press, William Pollock, 2010, ISBN: 978-1-59327-220-3.

[29] F. Z. Boito, R. Nou, L. L. Pilla, *et al.*, „On server-side file access pattern matching“, Jul. 2019. DOI: 10.1109/HPCS48598.2019.9188092.

[30] T. Ilsche, J. Schuchart, R. Schöne, and D. Hackenberg, „Combining instrumentation and sampling for trace-based application performance analysis“, in Jan. 2015, pp. 123–136, ISBN: 978-3-319-16011-5. DOI: 10.1007/978-3-319-16012-2_6.

[31] GeeksforGeeks, *Interval tree | set 1 (introduction)*, https://www.geeksforgeeks.org/interval-tree/, Accessed: 2025-03-20, 2025.

[32] C. Kingsford, *Interval trees*, https://www.cs.cmu.edu/~ckingsf/bioinfo-lectures/intervaltrees.pdf, Accessed: 2025-03-20, 2025.

[33] Baeldung, *Tree data structures for interval and range queries*, https://www.baeldung.com/cs/tree-segment-interval-range-binary-indexed, Accessed: 2025-03-20, 2025.

[34] P. E. Richard Gooch. „Overview of the linux virtual file system“. (2005), [Online]. Available: https://www.kernel.org/doc/html/v5.7/filesystems/vfs.html (visited on Mar. 17, 2025).

[35] M. Gorman, *Understanding the Linux Virtual Memory Manager*. Upper Saddle River, New Jersey 07458: Pearson Education, Inc., 2004, ISBN: 0-13-145348-3.

[36] SquaredUp, *Metrics vs logs vs traces vs profiles*, https://squaredup.com/blog/metrics-vs-logs-vs-traces-vs-profiles/, Accessed: 2025-03-18, 2024.

[37] G. Labs, *Grafana tempo documentation*, https://grafana.com/docs/tempo/latest/, Accessed: 2025-03-18, 2024.

[38] CoderSociety, *Jaeger vs zipkin vs tempo: A comparison of distributed tracing tools*, https://codersociety.com/blog/articles/jaeger-vs-zipkin-vs-tempo, Accessed: 2025-03-18, 2024.

[39] P. Authors, *Prometheus - an introduction and overview*, https://prometheus.io/docs/introduction/overview/, Accessed: 2025-03-18, 2024.

[40] Logz.io, *A guide to prometheus metrics*, https://logz.io/learn/prometheus-metrics-guide/, Accessed: 2025-03-18, 2024.

[41] M. Wallschläger, A. Gulenko, F. Schmidt, A. Acker, and O. Kao, „Anomaly detection for black box services in edge clouds using packet size distribution“, Oct. 2018, pp. 1–6. DOI: 10.1109/CloudNet.2018.8549546.

[42] Y. Rabinovich. „Flora: The new ebpf observability sensor from groundcover“. (2025), [Online]. Available: https://www.groundcover.com/blog/ebpf-observability-agent (visited on Mar. 19, 2025).

[43] M. Craun, K. Hussain, U. Gautam, Z. Ji, T. Rao, and D. Williams, „Eliminating ebpf tracing overhead on untraced processes“, in *Proceedings of the ACM SIGCOMM 2024 Workshop on EBPF and Kernel Extensions*, ser. eBPF '24, Sydney, NSW, Australia: Association for Computing Machinery, 2024, 16–22, ISBN: 9798400707124. DOI: 10.1145/3672197.3673431. [Online]. Available: https://doi.org/10.1145/3672197.3673431.

[44] S. Snyder, P. Carns, K. Harms, R. Ross, G. K. Lockwood, and N. J. Wright, „Modular hpc i/o characterization with darshan“, in *2016 5th Workshop on Extreme-Scale Programming Tools (ESPT)*, 2016, pp. 9–17. DOI: 10.1109/ESPT.2016.006.

[45] O. Project, *Opentelemetry metrics sdk specification*, https://opentelemetry.io/docs/specs/otel/metrics/sdk/, Accessed: 2025-03-20, 2025.

[46] O. Project, *Transforming telemetry data with opentelemetry collector*, https://opentelemetry.io/docs/collector/transforming-telemetry/, Accessed: 2025-03-20, 2025.

[47] P. Signals, *Profiling python and ruby with ebpf*, https://www.polarsignals.com/blog/posts/2023/10/04/profiling-python-and-ruby-with-ebpf, Accessed: 2025-03-20, 2023.

[48] G. Labs, *Tempo backend search operations*, https://grafana.com/docs/tempo/latest/operations/backend_search/, Accessed: 2025-03-20, 2025.

[49] G. Labs, *Tempo caching mechanism*, https://grafana.com/docs/tempo/latest/operations/caching/, Accessed: 2025-03-20, 2025.

[50] G. Labs, *Sizing guidelines for grafana tempo*, https://grafana.com/docs/tempo/latest/setup/size/, Accessed: 2025-03-20, 2025.

Appendices

A Original project description

Fachhochschule Nordwestschweiz

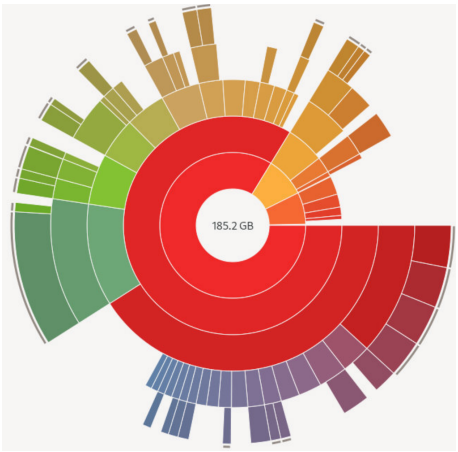
HW | Sys Sec

24HS_I4DS28: Entwicklung eines spezialisierten eBPF-basierten Profilers für Dateizugriffe in HPC-Umgebungen

			Priorität 1	Priorität 2
Betreuer:	André Csillaghy Simon Marcin	**Arbeitsumfang:**	P5 oder P6	---
		Teamgrösse:	2er Team	---
Sprachen:	Deutsch oder Englisch			
Studiengang:	Informatik			

Ausgangslage

Im Bereich des High-Performance Computing (HPC) ist die Effizienz von Dateizugriffen ein kritischer Faktor für die Leistung wissenschaftlicher Anwendungen. Viele Workflows skalieren nicht gut auf grossen HPC-Systemen, da die CPUs nicht schnell genug mit Daten versorgt werden können. Insbesondere bei massiv parallelen Zugriffsoperationen auf ein geteiltes Dateisystem treten Engpässe auf, die die Gesamtleistung erheblich beeinträchtigen können. Moderne Profiling-Technologien, insbesondere solche, die auf Extended Berkeley Packet Filter (eBPF) basieren, bieten neue Möglichkeiten, detaillierte Einblicke in Systemoperationen zu erhalten. Ein spezialisierter eBPF-basierter Profiler könnte dabei helfen, detaillierte Informationen über alle Dateizugriffe zu sammeln und Engpässe zu identifizieren.



Visualisierung von Dateizugriffen über ein hierarchisches Dateisystem verteilt

Ziel der Arbeit

Das Ziel dieser Arbeit ist es, einen eigenen eBPF-basierten Profiler zu entwickeln, der sich auf Dateizugriffe spezialisiert. Dieser Profiler soll ein detailliertes Gesamtbild aller Dateizugriffe, einschliesslich Lese-, Schreib- und Speicherabbildungsoperationen, erstellen. Damit sollen Bottlenecks erkannt und die Effizienz der Dateizugriffe in HPC-Umgebungen verbessert werden. Ein weiteres Ziel ist die Visualisierung der Zugriffe auf einzelne Dateien, um darzustellen, wie und wo in den Dateien intensiv zugegriffen wurde. Diese Informationen sollen helfen, die Skalierbarkeit wissenschaftlicher Workflows auf grossen HPC-Systemen zu verbessern.

Problemstellung

1. Wie kann ein eBPF-basierter Profiler für Dateizugriffe mit minimalem Overhead entwickelt werden?
2. Welche Herausforderungen gibt es bei der Erfassung und Analyse von Dateizugriffen in HPC-Umgebungen?
3. Wie können die Daten visualisiert werden, um Dateizugriffe und ihre Auswirkungen darzustellen?
4. Wie können die gewonnenen Erkenntnisse genutzt werden, um die Skalierbarkeit und Effizienz wissenschaftlicher Workflow zu verbessern?

Bei einer Durchführung als IP6 zusätzlich:

5. Welche Zugriffsmuster lassen sich durch die Analyse von Dateizugriffen identifizieren?
6. Wie beeinflussen verschiedene Dateiformate (HDF5, FITS, ...) die Zugriffsmuster in HPC-Umgebungen?

Bemerkung

Das Institut für Data Science (I4DS) hat das HPC-Framework für die ESA-Mission Euclid entworfen, welches derzeit aktiv auf neun HPC-Clustern weltweit genutzt wird. Der geplante PoC soll mit echten Workflows der Euclid-Mission getestet werden. Zusätzlich tritt das Paul Scherrer Institut (PSI) als weiterer Forschungspartner und Kunde auf. Der PoC wird ebenfalls auf deren HPC-Infrastruktur getestet, um die Anwendbarkeit und Leistungsfähigkeit der Profiler in verschiedenen HPC-Umgebungen zu evaluieren. Diese Zusammenarbeit soll sicherstellen, dass die entwickelten Lösungen praxisrelevant sind und den Anforderungen sowohl der Euclid-Mission als auch der wissenschaftlichen Projekte am PSI gerecht werden.

B Project declaration

1

Project declaration - 24HS_I4DS28

1 Problem Definition

Data analysis for ESA's Euclid mission requires High Performance Computing (HPC) due to the large amount of gathered data.[1] The Institute for Data Science (I4DS) at the University of Applied Sciences and Arts, Northwestern Switzerland (FHNW) aims to improve the performance in scientific code. File access is often one of the major bottlenecks in a shared file system. The aim is to develop a tool to analyze and optimize file access performance. Currently, the code seems to work on regular computers and on local clusters with up to a few hundred CPU cores. However, when the code is run on Supercomputers providing over 2000 CPU cores, the file system experiences breakdowns.

An eBPF-based profiler for file accesses should be developed in order to examine the cause of the aforementioned issues. It should provide insights into what files have been accessed by the code, how often and by what process they have been opened. Furthermore, results should contain the access patterns such that one can determine if a different file format could improve performance. The profiler should highlight problematic lines of code to guide necessary changes.

In the HPC environment, multiple servers might run the same workload in parallel and the results of all the nodes must be aggregated into a single output.

The role of visualization is very important in this project. It enables key questions to be answered, including identifying the most important metrics, determining the level of detail required by scientists, and the best way to present this information graphically. These considerations are made with the goal in mind to effectively communicate key data to the user for the purpose of performance optimization including frequency of file access, process details, and access patterns.

2 State of Research

Modern High-Performance Computing (HPC) systems are difficult to debug and troubleshoot because not much information can be retrieved from the kernel. These systems are characterized by their immense scale and the multitude of software they run. The HPC software targeted by this project cannot be isolated or managed using containerization technologies like Docker or traditional virtualization. This is due to the software's need for direct hardware access and its complex execution across multiple nodes. Thus, alterantive grouping and aggregation approaches must be considered [2].

eBPF tools are employed for the analysis and overviewing of system calls in the kernel and user space. eBPF attaches probes to kernel and user space system calls. These probes provide access to kernel and user routines without interrupting ongoing processes. The BPF Compiler Collection is a set of tools that makes it possible to analyze kernel and user space software using tracing, sampling, and snooping [3]. eBPF is widely used in cloud environments. AWS, Apache, and Grafana describe how they have used eBPF in their products. The fundamental concept is analogous to that of this project, although the operational context differs. The system that will run this project's profiler hosts a variety of programs and applications. The products of other vendors are configured for specific programs in cloud environments, e.g., Kubernetes. Another example of eBPF usage is Cilium. It is based on eBPF. Google describes it as follows: "Cilium is an open-source project that has been designed on top of eBPF to address the new scalability, security, and visibility requirements of container workloads. Cilium goes beyond a traditional

Container Networking Interface (CNI) to provide service resolution, policy enforcement, and much more [4]."

Visualizing eBPF metrics for file access in HPC environments is critical to this project's success. However, there is a dearth of existing projects that directly address the specific needs of such environments. The open-source software Grafana is widely used for monitoring system metrics, including file access patterns. However, current solutions like the "Grafana Kernel eBPF Hook" and "Beyla" dashboards focus on general system call visualization and lack tailored solutions for the specific file system issues that arise in HPC environments. Furthermore, while these solutions are capable of tracking file usage and I/O performance, they are primarily designed for cloud environments and containerized workloads, rather than the large-scale, non-containerized scientific codes that are the focus of this project.[5][6][7][8]

3 Problem and Research Questions

Questions to be answered during research or implementation of the solution include:

- Is it better to use instrumented profiling or sampling-based profiling?
  - Instrumented profiling is triggered e.g. on each read-operation
  - Sampling-based profiling is triggered periodically, e.g. at a frequency of 50Hz
- What inefficient file access patterns exist?
  - How can they be detected using eBPF?
  - Are there file access patterns unique to HPC environments, given their vastly different hardware and software configurations?
- How does the solution handle access to RAM, SSD, HDD, and sharedFS with mmap, considering caching, network overhead, and transfer bandwidth?
- What are the most efficient methods for consolidating distributed profiling data across nodes?
  - Runs of the same executable across multiple nodes should result in one single output
  - These runs may occur over an extended period, complicating the aggregation and consolidation process.
- What Linux Kernel version is needed to support the required eBPF-features?
- How large is the CPU overhead generated by the solution?
  - Is it low enough such that the profiler could run always-on in production?
  - Will need to be measured using appropriate tooling
- How should the data be represented?
  - Which metrics are recorded?
  - What information do the scientists need?
  - What graphical representations are helpful?

4 Solution Approach

As in every eBPF application, there will be a Kernel-space program (KSP) and a User-space program (USP). But given the performance-critical nature of this profiler, a third layer is introduced:

- Kernel-space program (KSP)
  - collects data, triggered by system calls, stores data in BPF maps
- User-space program (USP)
  - loads and interacts KSP
  - retrieves data from BPF maps and stores it either locally or in a DB
- Visualization layer
  - visualizes the data acquired during profiling

This solution benefits from the visualization capabilities of the technology used in the visualization layer while ensuring no additional performance overhead is generated by keeping it separate from the USP. Additionally, this solution can leverage existing technologies or frameworks, such as a pre-built visualization module, if applicable.

The approach is to develop a profiler, that tracks file accesses by certain jobs, projects or other aggregation dimensions that can be activated.

The profiling should - if feasible and beneficial for this use case - use formats and standards defined in the OpenTelemetry continuous profiling agent [9].

Currently, the technologies for each layer are undecided. Part of the project involves determining the essential metrics and visualization techniques required to help scientists improve their code, which will then guide technology selection.

5 Risk assessment

Risk	Impact	Probability	Mitigation
Overhead on HPC workloads	High	Low	Measure overhead, optimize probes
Permissions or security restrictions	High	Medium	Coordinate with admins early
Visualization performance issues	Medium	High	Optimize aggregation and rendering
Data unusable for visualization	Medium	Medium	Define schema early, test early
Delayed access to HPC resources	High	Medium	Use local/virtual testing first
Insufficient time for development	High	Medium	Prioritize work packages

6 Timeline

2024
2025
Sep
Oct
Nov
Dec
Jan
Feb
Mar
38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 1 2 3 4 5 6 7 8 9 10 11 12
Milestones
Input PSI
Begin development
Backend working
Report Hand-in
Research
State of Research
File access patterns
Software architecture
Software & Testing
KSP&USP setup
Data Flow
Detect patterns
Highlight Lines of Code
Plotting
Documentation
Project declaration
Introduction
State of Research
Theory
Implementation
Conclusion
Final touches

Bibliography

[1] ESA. „Esa - euclid overview“. (Sep. 17, 2024), [Online]. Available: https://www.esa.int/Science_Exploration/Space_Science/Euclid_overview (visited on Sep. 17, 2024).

[2] W. Cadena, A. Ng, and M. Phillips, *Bcc tools - info poster*, https://www.lanl.gov/org/ddste/aldsc/hpc/_assets/docs/posters/BCC-tools-Info-poster.pdf, Mentors: Travis Cotton, Nick Jones, Johnathan Nielsen. Accessed: 24-Sep-2024, Los Alamos National Laboratory, 2024.

[3] M. A. Phillips, A. M. Ng, and W. J. Cadena, *Extended berkeley packet filter (ebpf) - an introduction*, https://www.lanl.gov/org/ddste/aldsc/hpc/_assets/docs/mini-showcase-abstracts/ExtendedBerkeleyPacketFilter.pdf, Accessed: 24-Sep-2024, Los Alamos National Laboratory, 2024.

[4] G. Johar and V. Marupadi, *Bringing ebpf and cilium to google kubernetes engine*, https://cloud.google.com/blog/products/containers-kubernetes/bringing-ebpf-and-cilium-to-google-kubernetes-engine?hl=en, Accessed: 24-Sep-2024, Google Cloud, 2024.

[5] R. R. M. T. Nikola Grcevski Mario Macías, *Beyla - zero code instrumentation using ebpf*, https://grafana.com/oss/beyla-ebpf/, Accessed: 14-Oct-2024, Grafana Labs, 2024.

[6] G. Labs, *Kernel ebpf hook - grafana dashboard*, https://grafana.com/grafana/dashboards/18612-kernel-ebpf-hook/, Accessed: 14-Oct-2024, Grafana Labs, 2024.

[7] N. Grcevski, *Grafana beyla 1.0 release: Zero code instrumentation for application telemetry using ebpf*, https://grafana.com/blog/2023/11/14/grafana-beyla-1.0-release-zero-code-instrumentation-for-application-telemetry-using-ebpf/, Accessed: 14-Oct-2024, Grafana Labs, 2023.

[8] Komprise, *Storage metrics - glossary term*, https://www.komprise.com/glossary_terms/storage-metrics/, Accessed: 14-Oct-2024, Komprise, 2024.

[9] OpenTelemetry. „Opentelemetry - elastic contributes continuous profiling agent“. (Oct. 12, 2024), [Online]. Available: https://opentelemetry.io/blog/2024/elastic-contributes-continuous-profiling-agent (visited on Oct. 12, 2024).